Problem 1:

Create a simple Node.js HTTP server that listens on port 3000 and routes requests based on the URL path. The server should handle the following:

- return "Welcome to the homepage!"
- about should return "This is the about page."
- Any other URL should return "Page not found" with a 404 status.

Server1.js

```
const http = require("http");

const hostname='127.0.0.1';

const port=3000;


const server = http.createServer((req, res) => {

    res.setHeader("Content-Type", "text/plain");

    if (req.url === "/") {

        res.statusCode = 200;

        res.end("Welcome to the homepage!");

    } else if (req.url === "/about") {

        res.statusCode = 200;

        res.end("This is the about page.");

    } else {

        res.statusCode = 404;

        res.end("Page not found");

    }

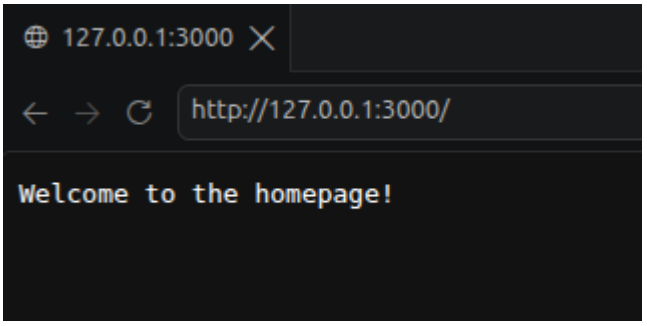
});


server.listen(port,hostname, () => {

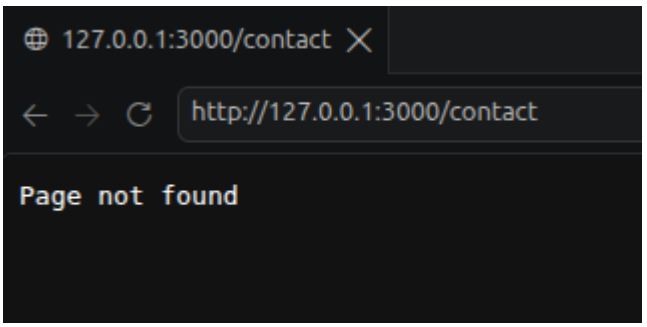
    console.log(`Server is running at
http://${hostname}:${port}/`);

});
```

```
student@nmamit-nitte:~/MERN$ node Server1.js
Server is running at  http://127.0.0.1:3000/
```







Problem 2:

Create a Node.js server to integrate of HTML tags and code directly within server-side handlers

Server2.js

```
const http = require('http');

const hostname='127.0.0.1';

const port=3000;

const server = http.createServer((req,res)=>{

    const url=req.url;

    if(req.url=="/")

    {

        res.writeHead(200,'Content-Type','text/plain');

        res.end(`

        <!DOCTYPE html>

        <html>

        <head><title>Sample-Page</title></head>

        <body>

        <h2>Hello NMAMIT</h2>

        <p>Welcome to Basics of MERN</p>

        </body>

        </html>`);

    }

    else if(req.url=="/about")

    {

        res.writeHead(200,'Content-Type','text/html');

        res.end(`<h1>About Page</h1>

                <p>This is a paragraph about-About</p>`);

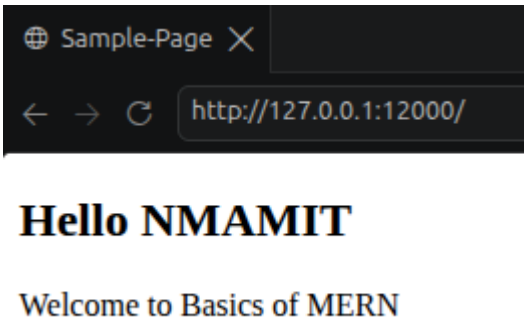
    }

    else if(req.url=="/help")
```

```
    {
        res.writeHead(200,'Content-Type','text/html');
        res.end(`<h1>Help</h1>
                <p>No-one is there to hel you?</p>`);
    }
    else
    {
        res.writeHead(200,'Content-Type','text/html');
        res.end(`<h1>Page not found</h1>`)
    }
});


    server.listen(port,hostname,() =>{
    console.log(`Server is running at
http://${hostname}:${port}/`);
});
```

```
student@nmamit-nitte:~/MERN$ node Server2.js
Server is running at  http://127.0.0.1:3000/
```





Problem 3:

Create a Node.js server that serves static files (HTML, CSS, JS) from the current directory. The server should return a proper 404 status code if the requested file doesn't exist.

(Place files: Server3.js, index.html, style.css, script.js in same folder)

Server3.js

```
const http = require("http");

const fs = require("fs");

const path = require("path");

const mimeTypes = {

    ".html": "text/html",

    ".css": "text/css",

    ".js": "application/javascript"

};


const server = http.createServer((req, res) => {

 let filePath = path.join(__dirname, req.url === "/" ? "index.html" : req.url);

 let ext = path.extname(filePath);

 let contentType = mimeTypes[ext] || "text/plain";

 fs.readFile(filePath, (err, data) => {

     if (err) {

            res.writeHead(404, { "Content-Type": "text/plain" });

            return res.end("404 File Not Found");

     }

     res.writeHead(200, { "Content-Type": contentType });

     res.end(data);

   });

});
```

```
server.listen(3000, () => {

    console.log("Static server running on
http://localhost:3000");

})
```

style.css

```
body {

    background-color: orange;

    font-family: Arial, Helvetica, sans-serif;

}
```

script.js

```
console.log("Hello From JavaScript");
```

index.html

```
<!DOCTYPE html>

<html>

<head>

  <title>Node Static Server</title>

  <link rel="stylesheet" href="style.css">

</head>

<body>

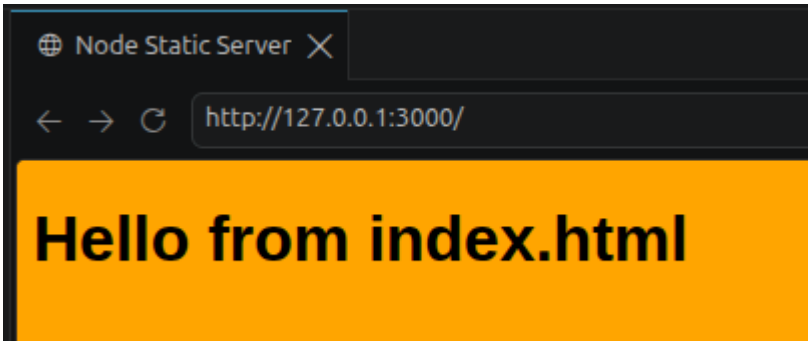
  <h1>Hello from index.html</h1>

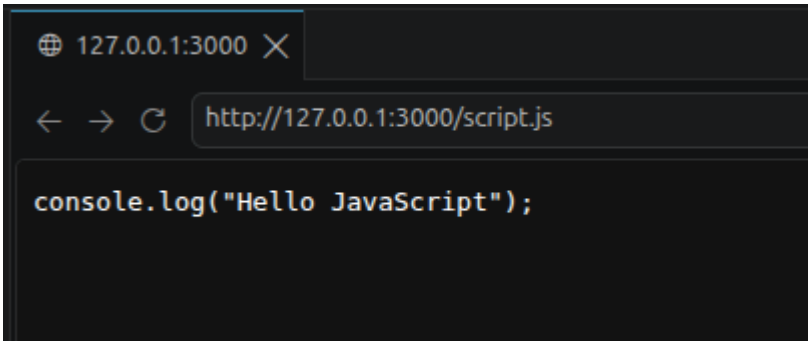
  <script src="script.js"></script>

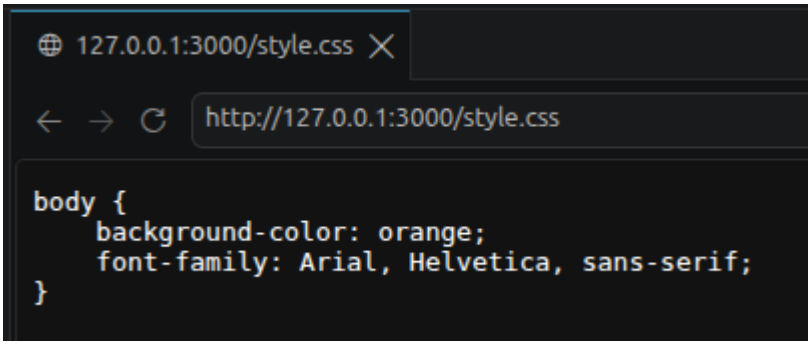
</body>

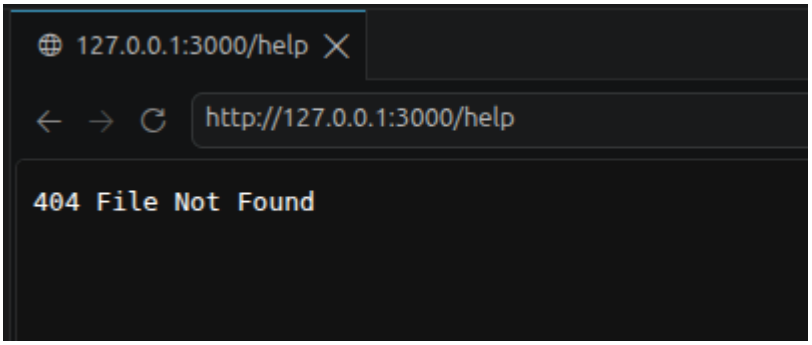
</html>
```

```
student@nmamit-nitte:~/MERN$ node server3.js
Server is running at  http://127.0.0.1:3000/
```









Problem 4:

Create a Node.js server to read and write JSON objects using files to save the json objects

Server4.js

```
const http=require("http");

const fs=require("fs");

const path=require("path");


const file=path.join(__dirname,'products.json')

const hostname='127.0.0.1';

const port=6000;


function getProducts(){

    if(!fs.existsSync(file)){

        return[];

    }

    const data=fs.readFileSync(file,'utf-8');

    return JSON.parse(data || "[]");

}


function saveProducts(products){

    fs.writeFileSync(file,JSON.stringify(products,null,2));

}


const server=http.createServer((req,res) =>

{

    const url=req.url;

    const method=req.method;

    res.setHeader('Content-Type','application/json')
```

```
 if(url==='/products' && method==='GET')
    {
        const products=getProducts();
        res.end(JSON.stringify(products,null,2))
    }
    else if(url==='/products' && method==='POST')
    {
        let body=""
        req.on("data",chunk =>{body+=chunk.toString()})
        req.on("end",()=>
        {
         try{
            const newProduct=JSON.parse(body)
            if(!newProduct.name || !newProduct.price)
            {
                 res.statusCode=400;
return res.end(JSON.stringify({error:"Both name and price are
required!"}));
             }
             const products=getProducts()
             products.push(newProduct)
             saveProducts(products)
             res.statusCode=201
res.end(JSON.stringify({message:"Products added successfully!"}));
             }
             catch(error)
             {
                 res.statusCode=400;
                 res.end(JSON.stringify({error:"Invalid JSON
data"}));
             }
        })
```

```
    }

    else

    {

        res.statusCode=404;

        res.end(JSON.stringify({error:"end/route point not
found"}));

    }

});


server.listen(port,hostname,()=>{

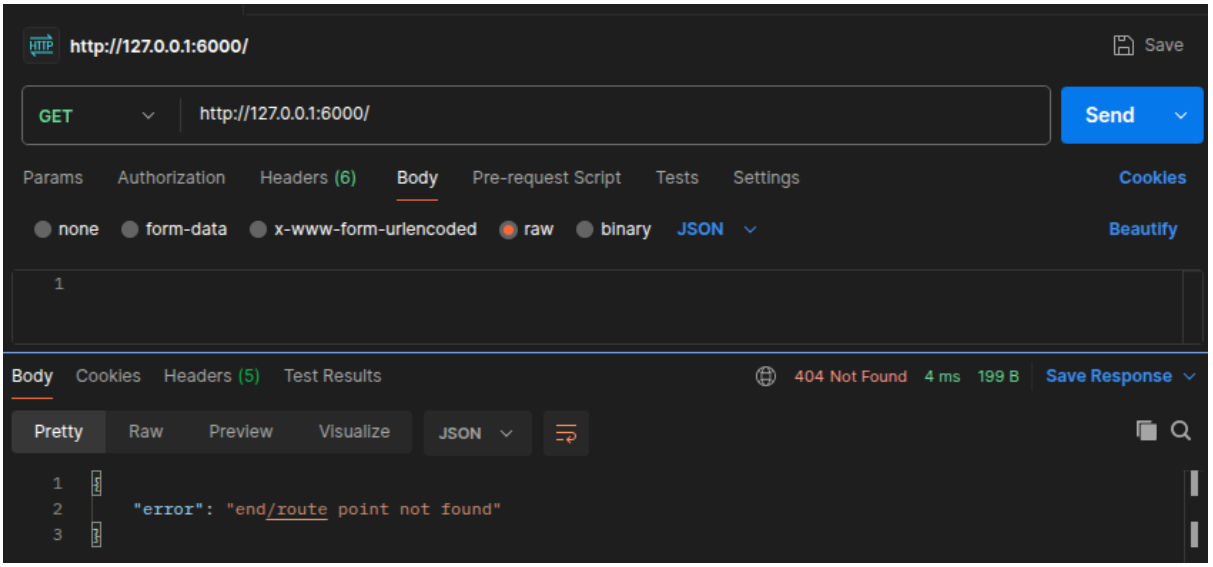
    console.log(`server running at  http://${hostname}:${port}/`)

});
```

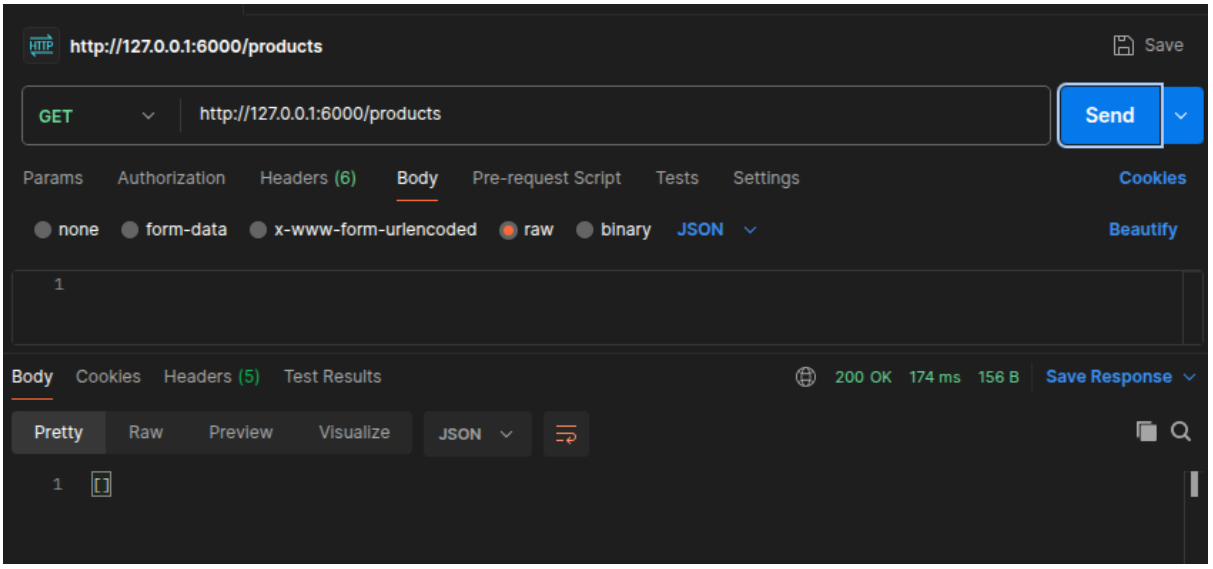
```
student@nmamit-nitte:~/MERN$ node server4.js
server running at  http://127.0.0.1:6000/
```

POSTMAN → Body → raw → Drop Down Select → JSON

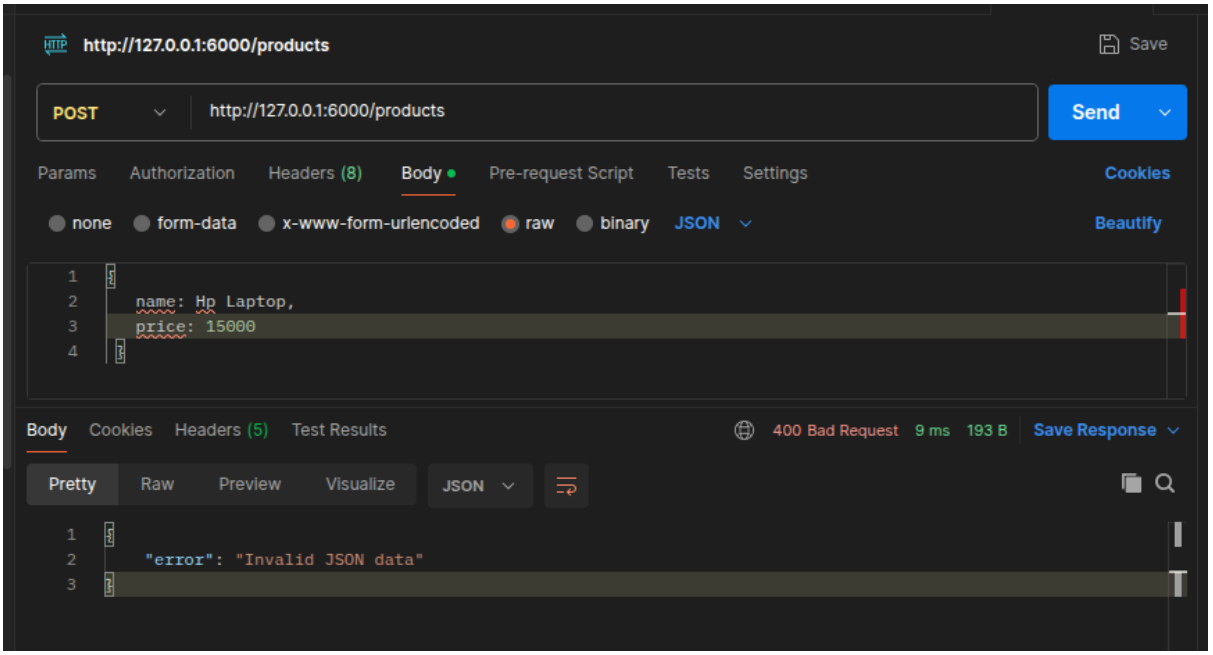
1. http://127.0.0.1:6000/ **Method → GET**



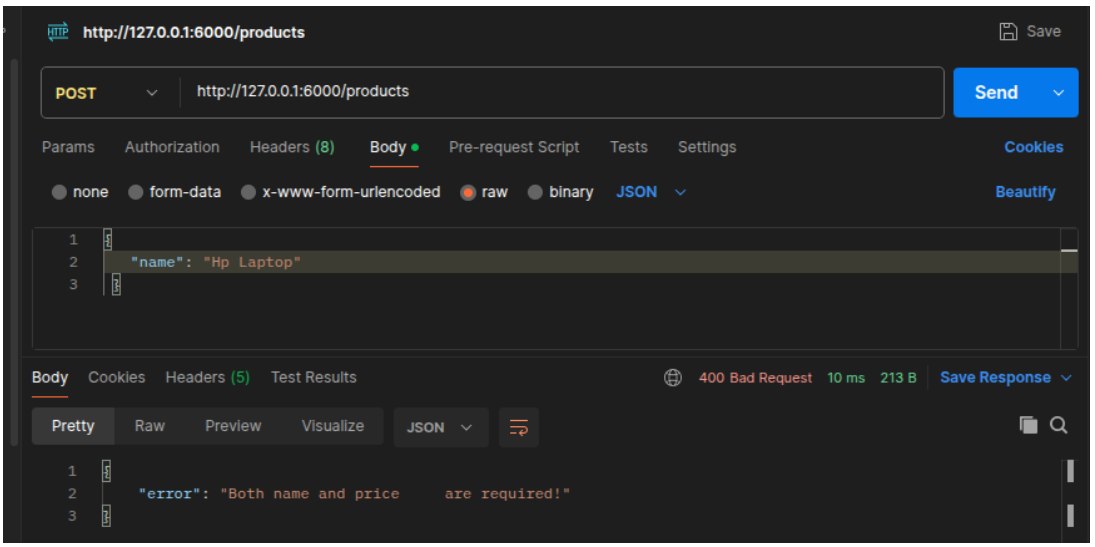
2. http://127.0.0.1:6000/products **Method → GET**



3. http://127.0.0.1:6000/products **Method → POST**



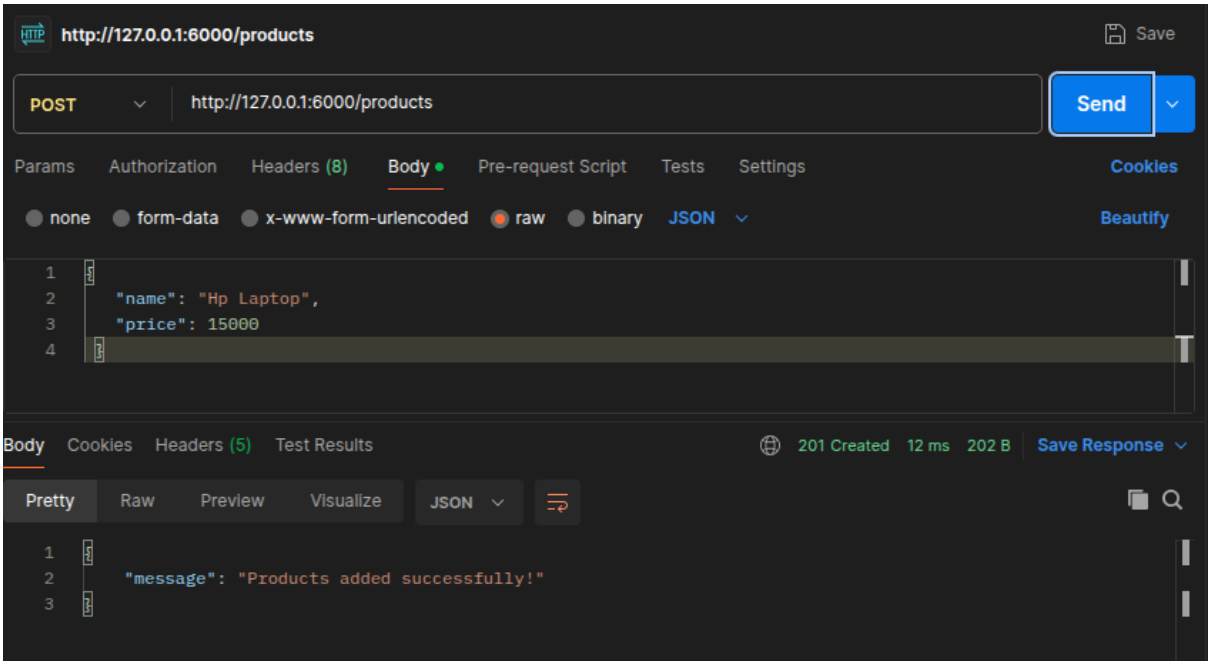
4. http://127.0.0.1:6000/products **Method → POST**



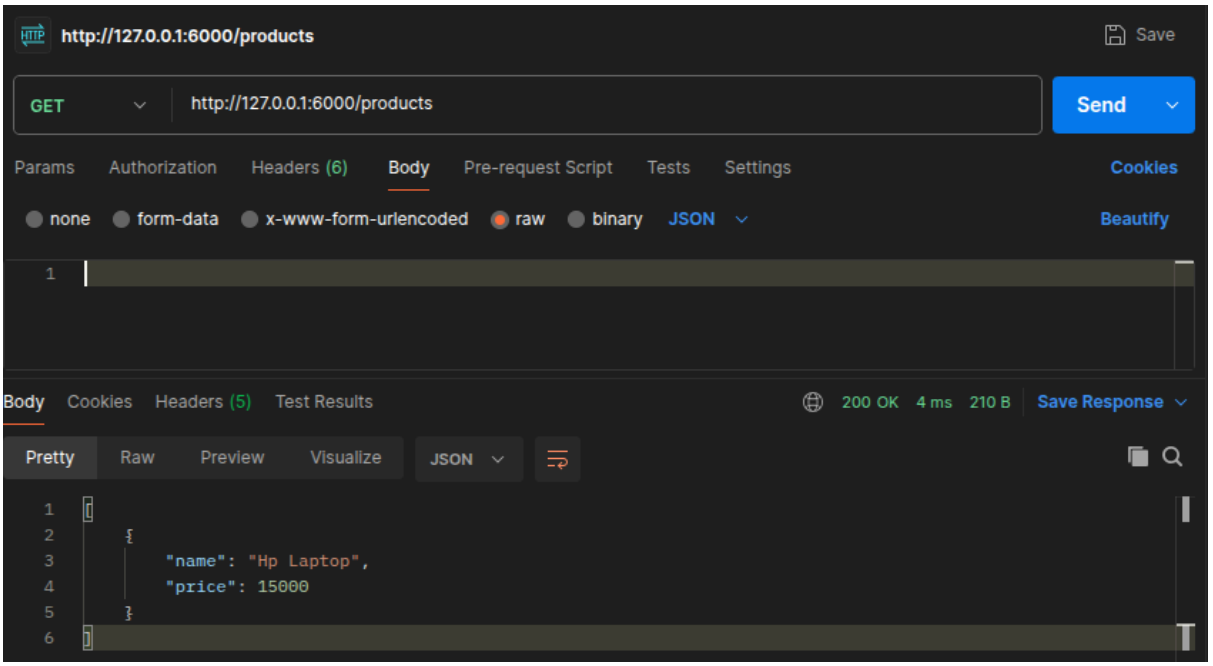
3. http://127.0.0.1:6000/products **Method → POST**

Add the data in Body

```
{
   "name": "Hp Laptop",
   "price": 15000
 }
```



4 http://127.0.0.1:6000/products **Method → GET**



Problem 5:

Create a REST API using Express.js. The API should support the following CRUD operations for managing name,description,price fields): items(using arrays & Item has

a. GET /items: Return a list of all items.

b. POST /items: Add a new item to the list.

c. PUT /items/:id: Update an item by ID.

d. DELETE /items/:id: Delete an item by ID.

Server5.js

```
const express = require("express");

const app = express();

app.use(express.json());

let items = [];

let id_counter = 1;


// CREATE

app.post("/product", (req, res) => {

    const { name, description, price } = req.body;

    let newItem = {

            id    : id_counter++,

            name  : name,

            description : description,

            price : price,

    };

    items.push(newItem);

    res.send(newItem);

});


// READ

app.get("/product", (req, res) => {

    res.json(items);
```

```
});

// UPDATE
app.put("/product/:id", (req, res) => {
    const { id } = req.params;
    const { name, description, price } = req.body;
    const index = items.findIndex((item) => item.id ===
parseInt(id));
    if (index === -1) {
        return res.send("Item not found");
    }

    items[index] = {
        id   : parseInt(id),
        name : name,
        description : description,
        price: price,
    };
    res.send("Updated Successfully");
});

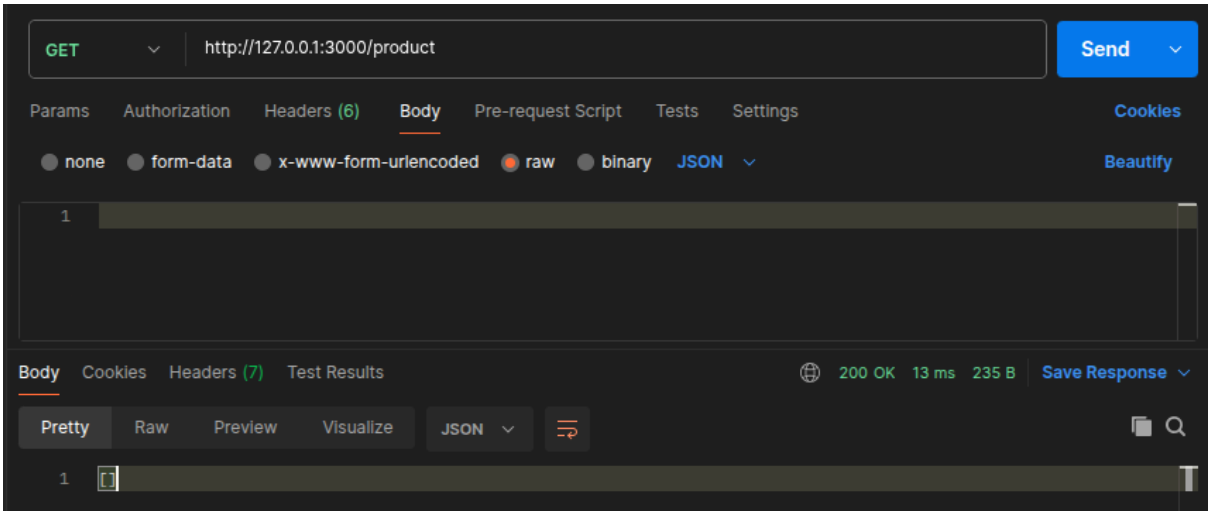
// DELETE
app.delete("/product/:id", (req, res) => {
    const { id } = req.params;
    const index = items.findIndex((item) => item.id ===
parseInt(id));
    if (index === -1) {
        return res.send("Item not found");
    }
    items.splice(index, 1);
    res.send("Deleted Successfully");
});
```

```
app.listen(3000, () => {
    console.log("Server started on port 3000");
});
```

```
student@nmamit-nitte:~/MERN$ node Server5.js
Server started on port 3000
```

POSTMAN → Body → raw → Drop Down Select → JSON

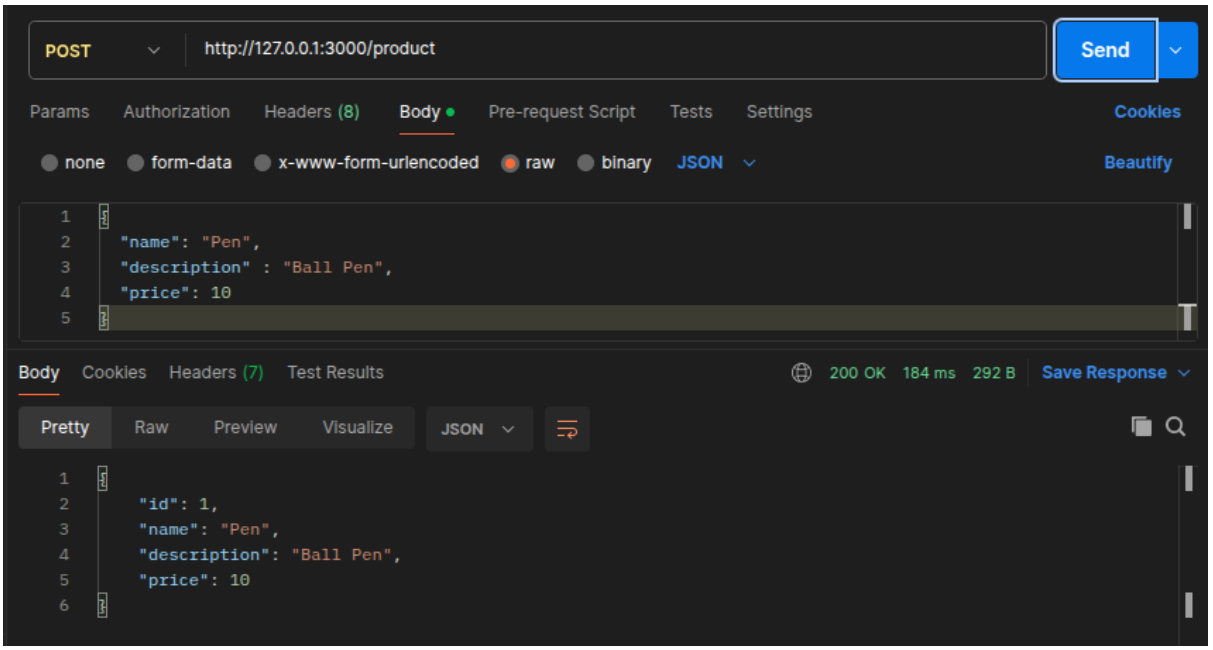
1. http://127.0.0.1:3000/product **Method → GET**



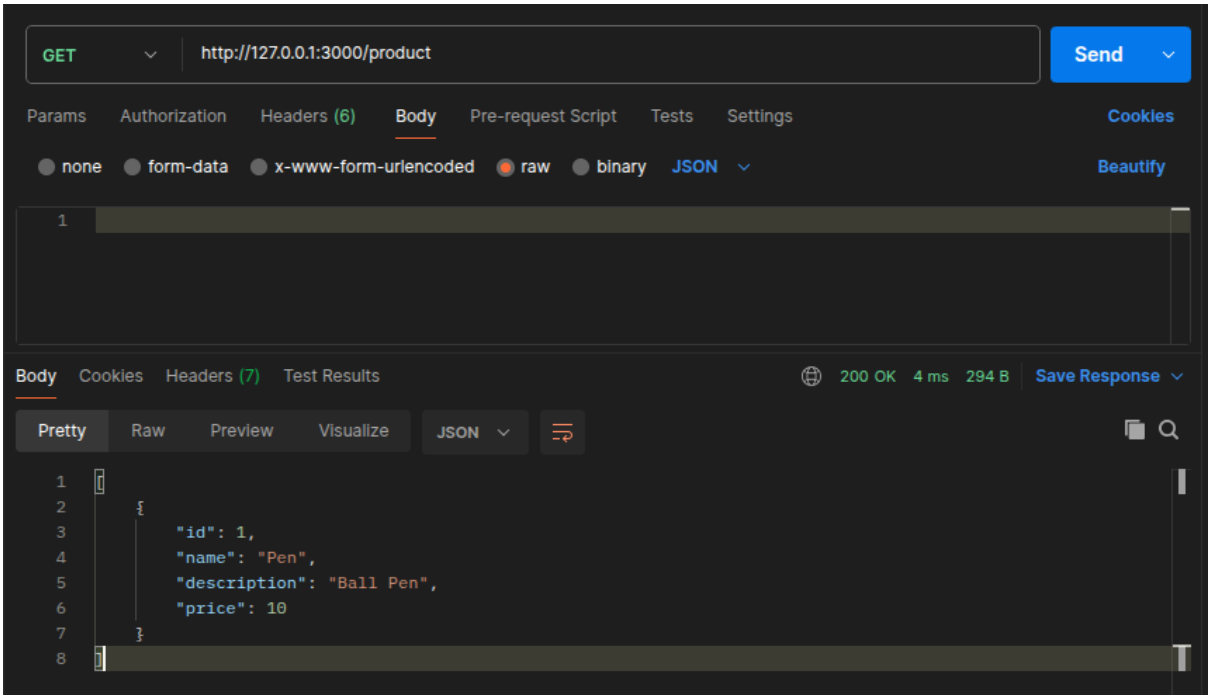
2. http://127.0.0.1:3000/product **Method → POST**

Add the data in Body

```
{
  "name": "Pen",
  "description" : "Ball Pen",
  "price": 10
}
```



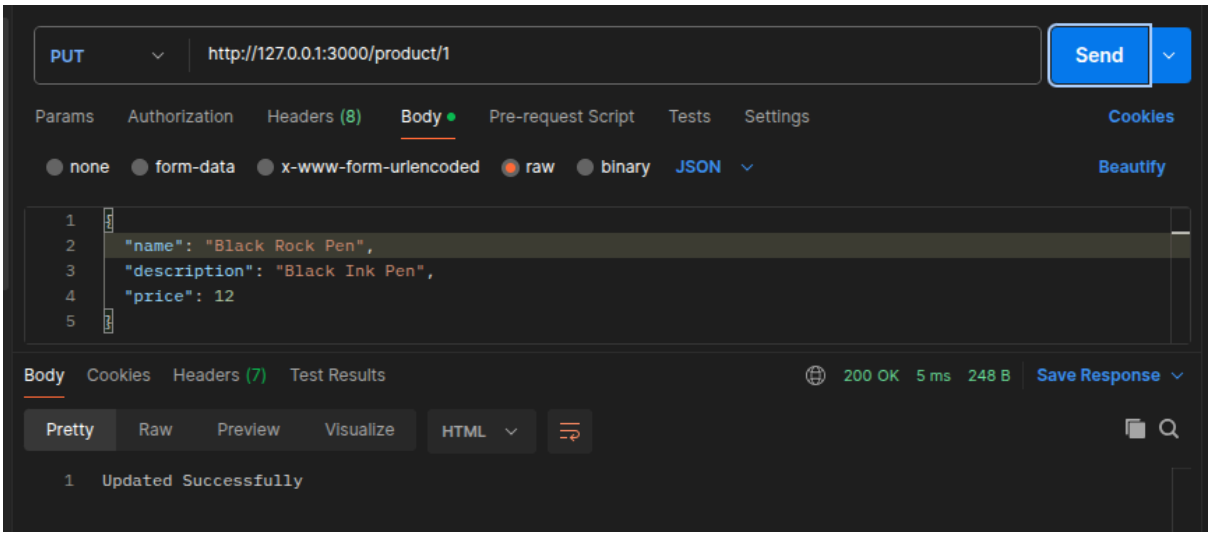
3. http://127.0.0.1:3000/product **Method → GET**



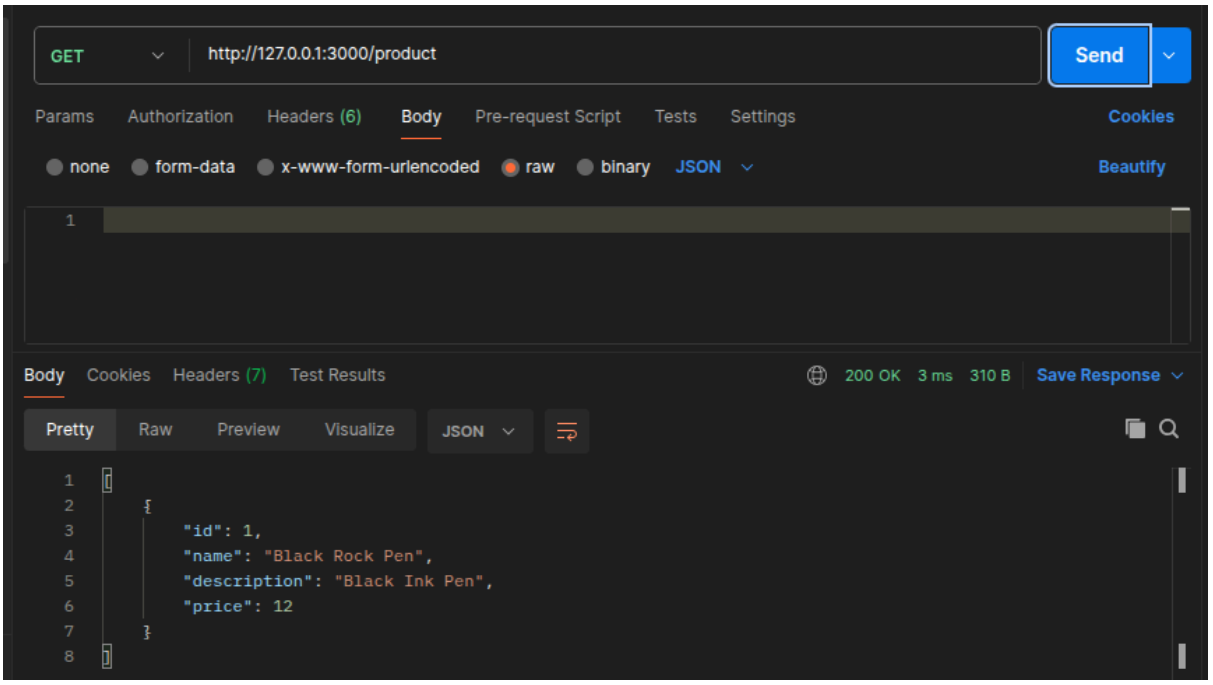
4. http://127.0.0.1:3000/product/1 **Method → PUT**

Add the updated data in Body

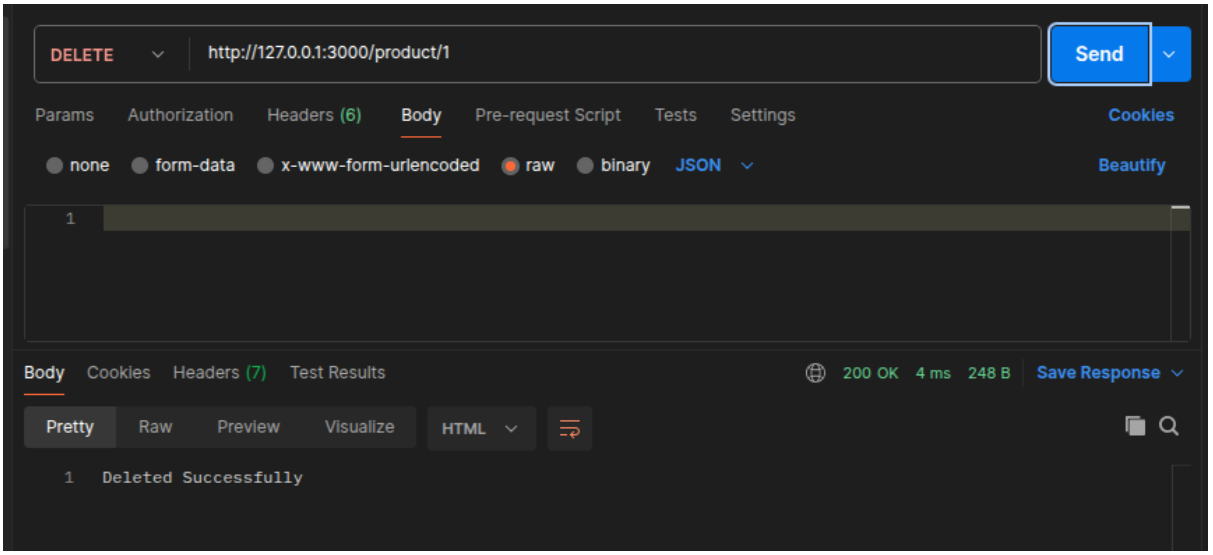
```
{
  "name": "Black Rock Pen",
  "description": "Black Ink Pen",
  "price": 12
}
```



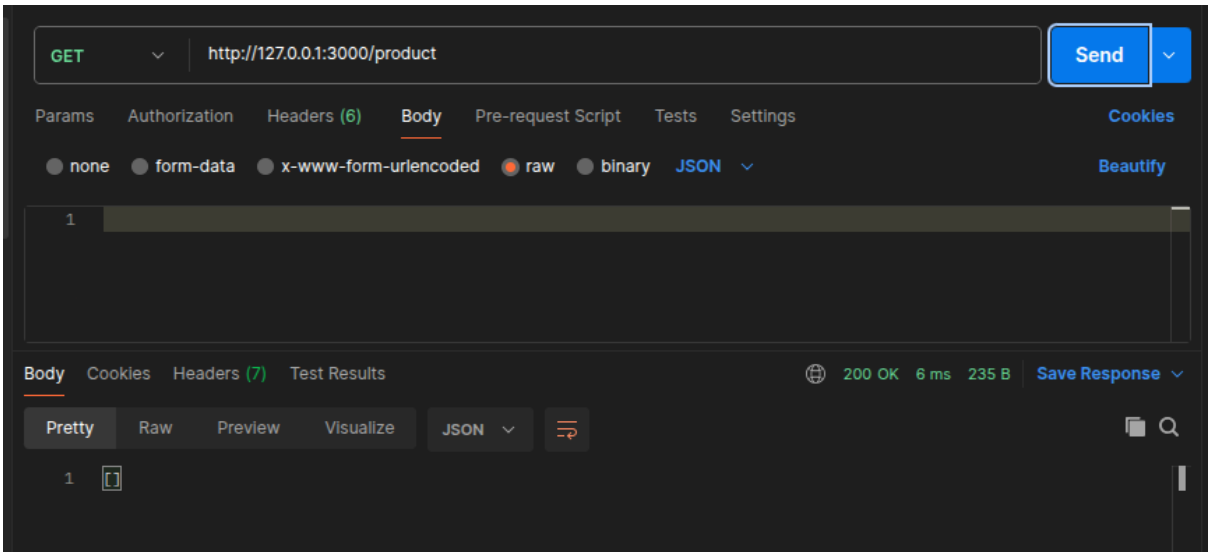
5. http://127.0.0.1:3000/product **Method → GET**



6. http://127.0.0.1:3000/product/1 **Method → DELETE**



7. http://127.0.0.1:3000/product **Method → GET**



Problem 6:

Create a REST API using Express.js. The API should support the following CRUD operations for managing items using files(save items in products.json). Item has name,description,price fields

a. GET /items: Return a list of all items.

b. POST /items: Add a new item to the list.

c. PUT /items/:id: Update an item by ID.

d. DELETE /items/:id: Delete an item by ID.

Server6.js

```
const express = require("express");

const fs = require("fs");

const path = require("path");

const app = express();

app.use(express.json());


const file = path.join(__dirname, "product.json");

counter = 0;


function read() {

    if (!fs.existsSync(file)) {

        return { counter: 0, items: [] };

    }

    const data = fs.readFileSync(file, "utf8");

    return JSON.parse(data || { counter: 0, items: [] });

}


function write(data) {

    fs.writeFileSync(file, JSON.stringify(data));

}
```

```
//CREATE
app.post("/product", (req, res) => {
    const { name, description, price } = req.body;
    let data = read();
    let newProduct = {
        id: ++data.counter,
        name: name,
        description : description,
        price: price,
    };
    data.items.push(newProduct);
    write(data);
    res.send(newProduct);
});


//READ
app.get("/product", (req, res) => {
    data = read();
    res.send(data.items);
});


//UPDATE
app.put("/product/:id", (req, res) => {
    const { id } = req.params;
    let { name, description, price } = req.body;
    let data = read();
    const index = data.items.findIndex((item) => item.id ===
parseInt(id));
    if (index == -1) {
```

```
        return res.send("Item not found");
    }
    data.items[index] = {
        id: parseInt(id),
        name: name,
        description : description,
        price: price,
    };
    write(data)
    res.send("Updated Successfully" );
});

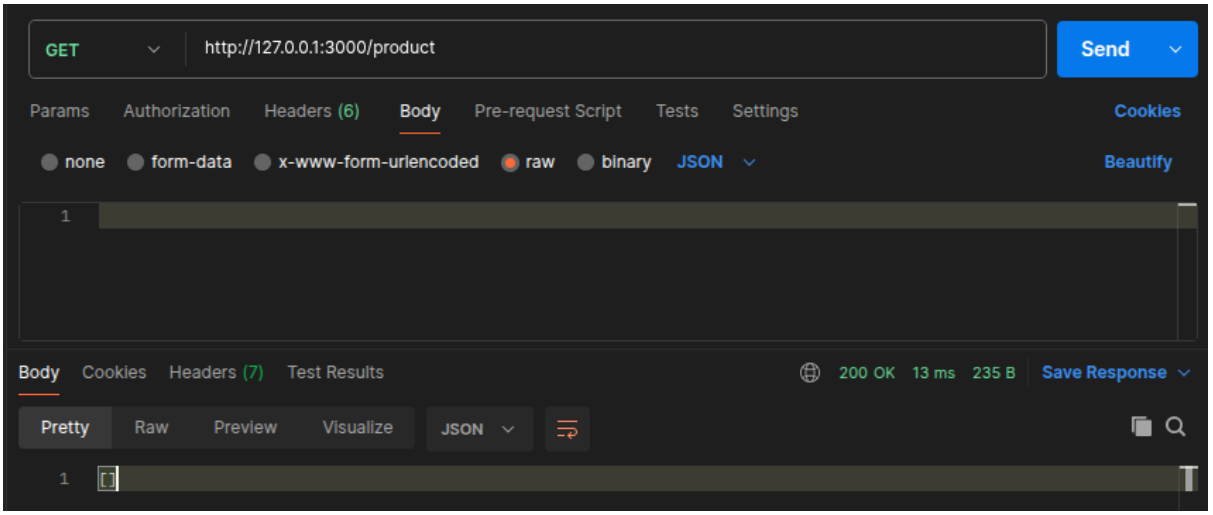

//DELETE
app.delete("/product/:id", (req, res) => {
    const { id } = req.params;
    let data = read();
    const index = data.items.findIndex((item) => item.id ===
parseInt(id));
    if (index === -1) {
        return res.send("Item not found");
    }
    data.items.splice(index, 1);
    write(data);res.send("Deleted Successfully");
});


app.listen(3000,()=>{
    console.log("Server started on port 3000");
});
```

```
student@nmamit-nitte:~/MERN$ node Server6.js
Server started on port 3000
```

POSTMAN → Body → raw → Drop Down Select → JSON

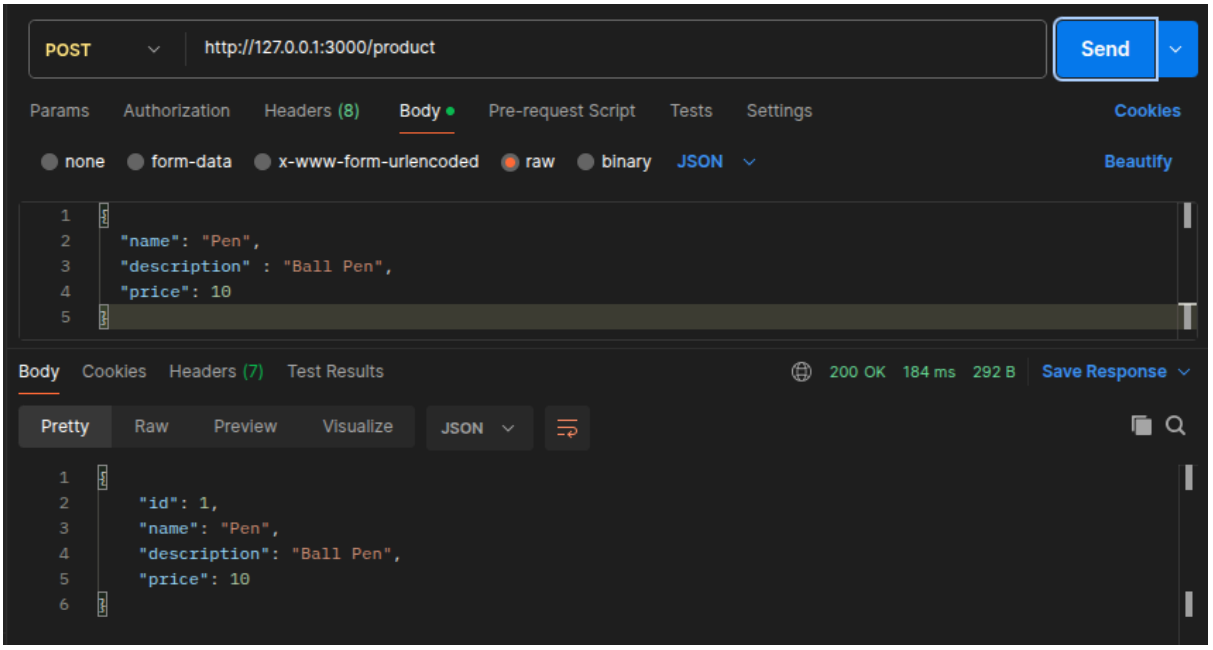
1. http://127.0.0.1:3000/product **Method → GET**



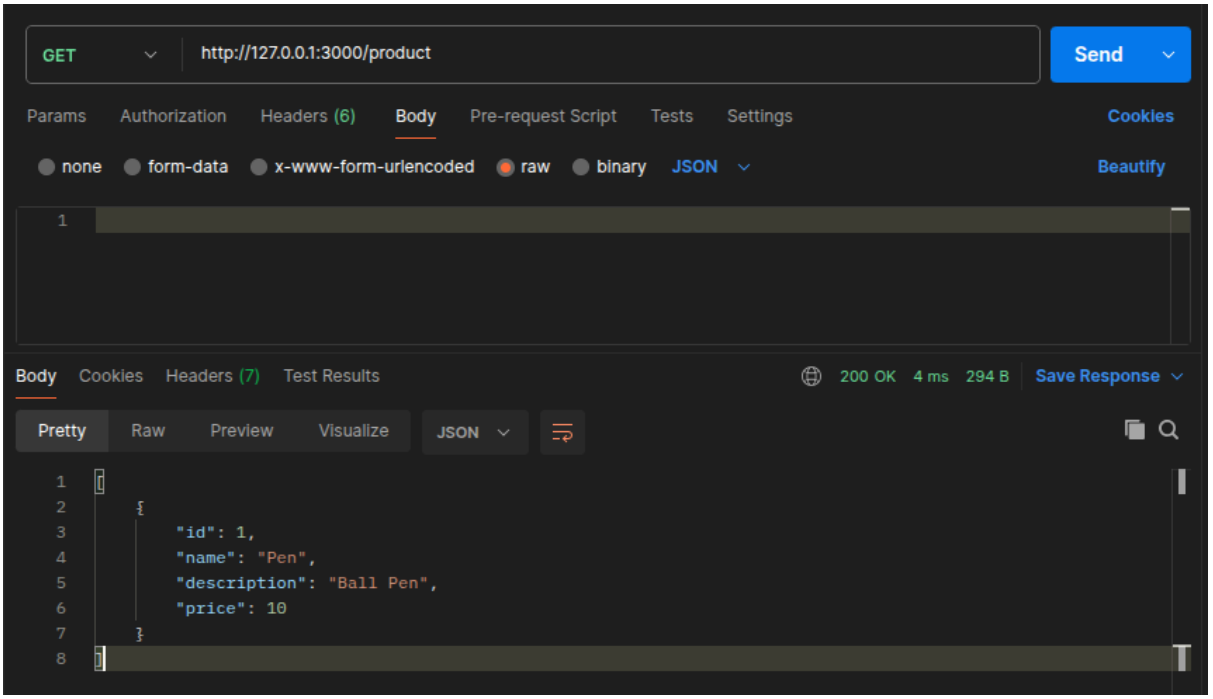
2. http://127.0.0.1:3000/product **Method → POST**

Add the data in Body

```
{
  "name": "Pen",
  "description" : "Ball Pen",
  "price": 10
}
```



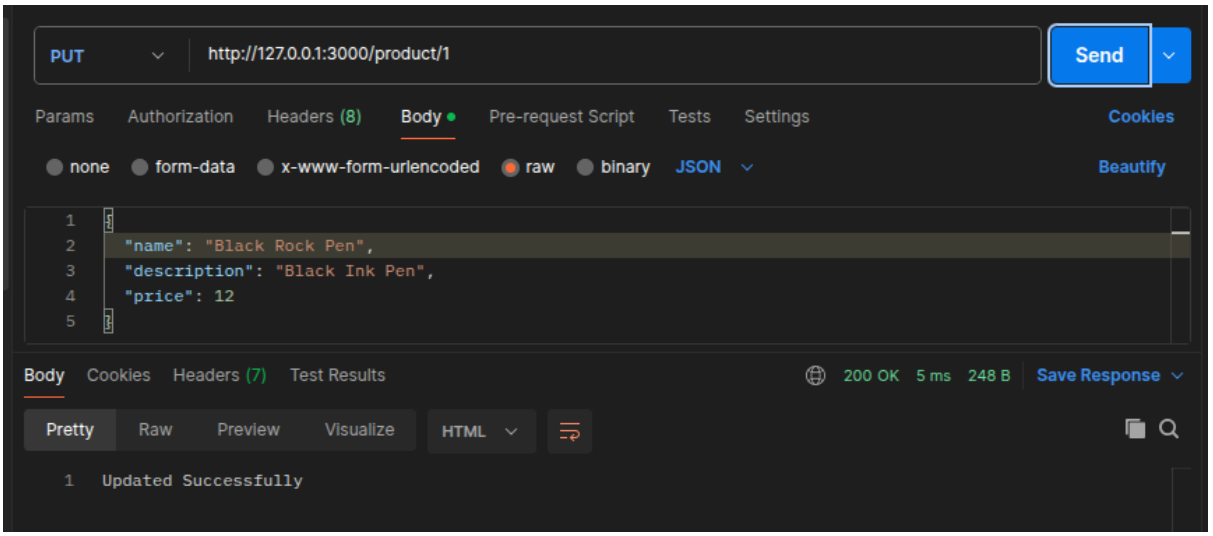
3. http://127.0.0.1:3000/product **Method → GET**



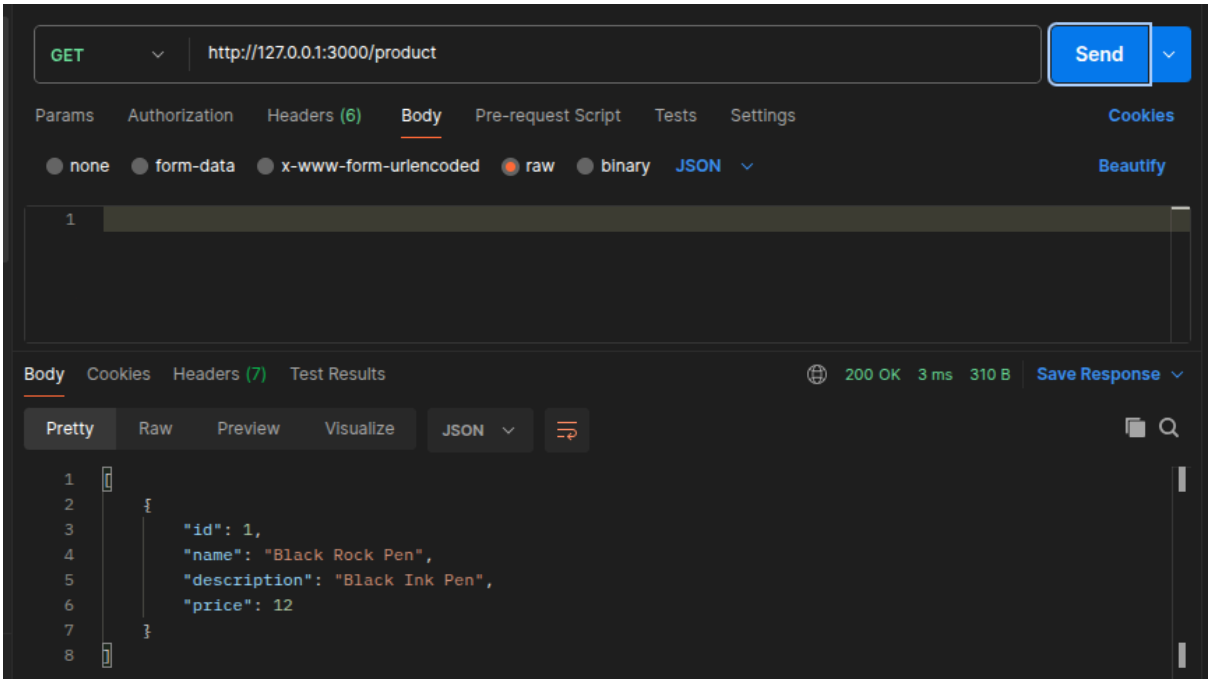
4. http://127.0.0.1:3000/product/1 **Method → PUT**

Add the updated data in Body

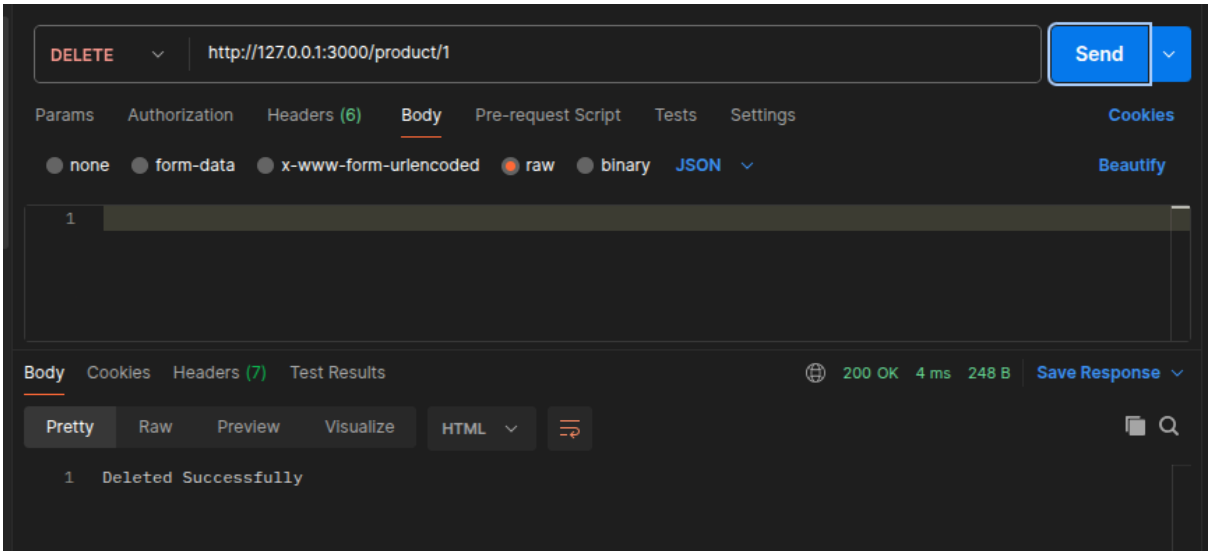
```
{
  "name": "Black Rock Pen",
  "description": "Black Ink Pen",
  "price": 12
}
```



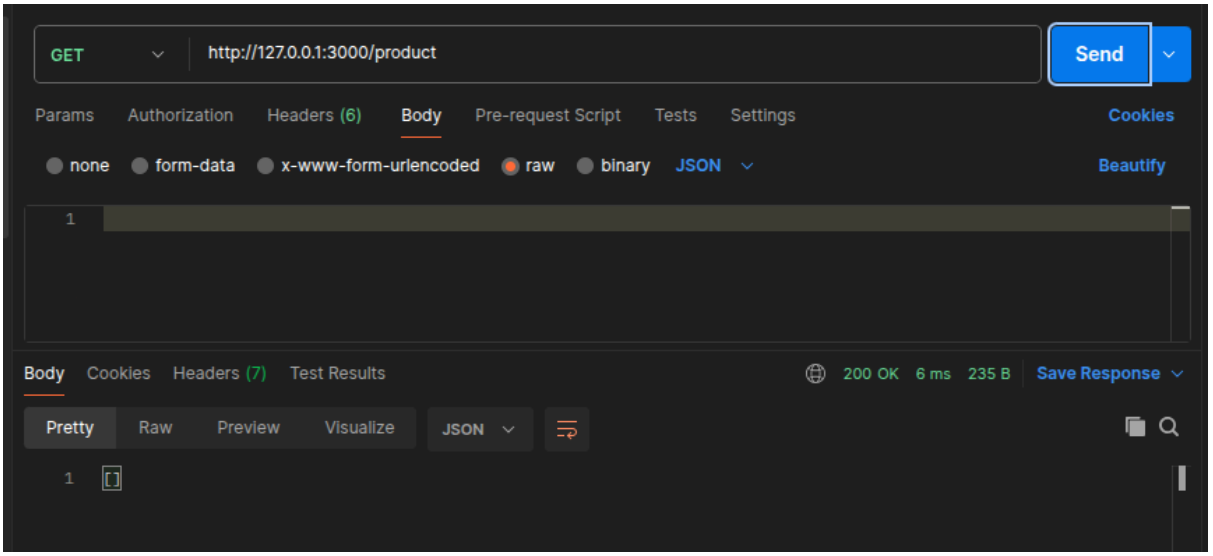
5. http://127.0.0.1:3000/product **Method → GET**



6. http://127.0.0.1:3000/product/1 **Method → DELETE**



7. http://127.0.0.1:3000/product **Method → GET**



Problem 7:

Create a simple REST API using Express.js and PostgreSQL that allows users to interact with a database. The API should support the following actions:

a. Create: Add a new user to the database.

b. Read: Retrieve all users from the database.

c. Update: Update a user's information in the database.

d. Delete: Delete a user from the database.

The database will store basic user information such as id, name, and email.

pgdb.js

```
const { Pool } = require('pg');

const pool = new Pool({

user: 'postgres',

password: 'admin',

host: 'localhost',

port: '5432',

database: 'MyDB',

});

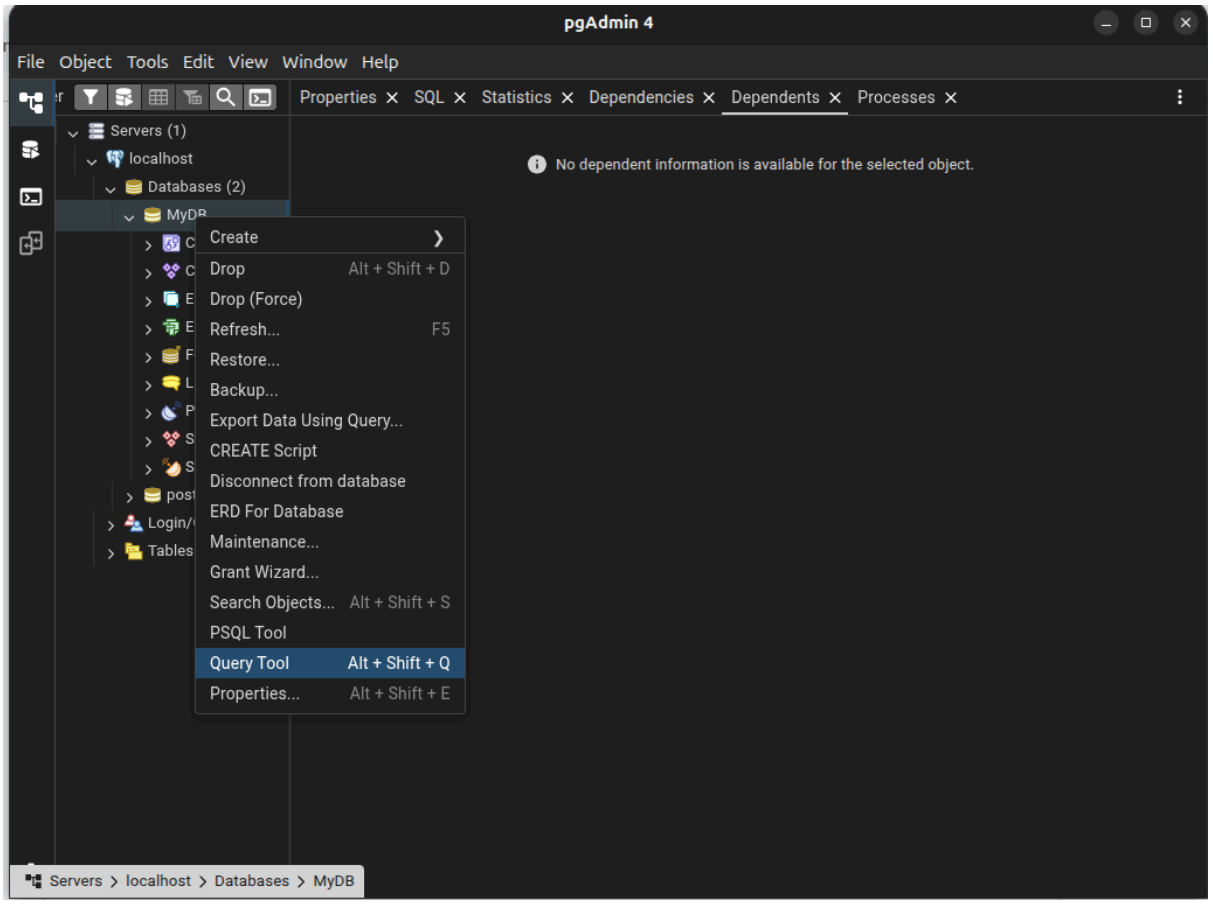
module.exports=pool;
```

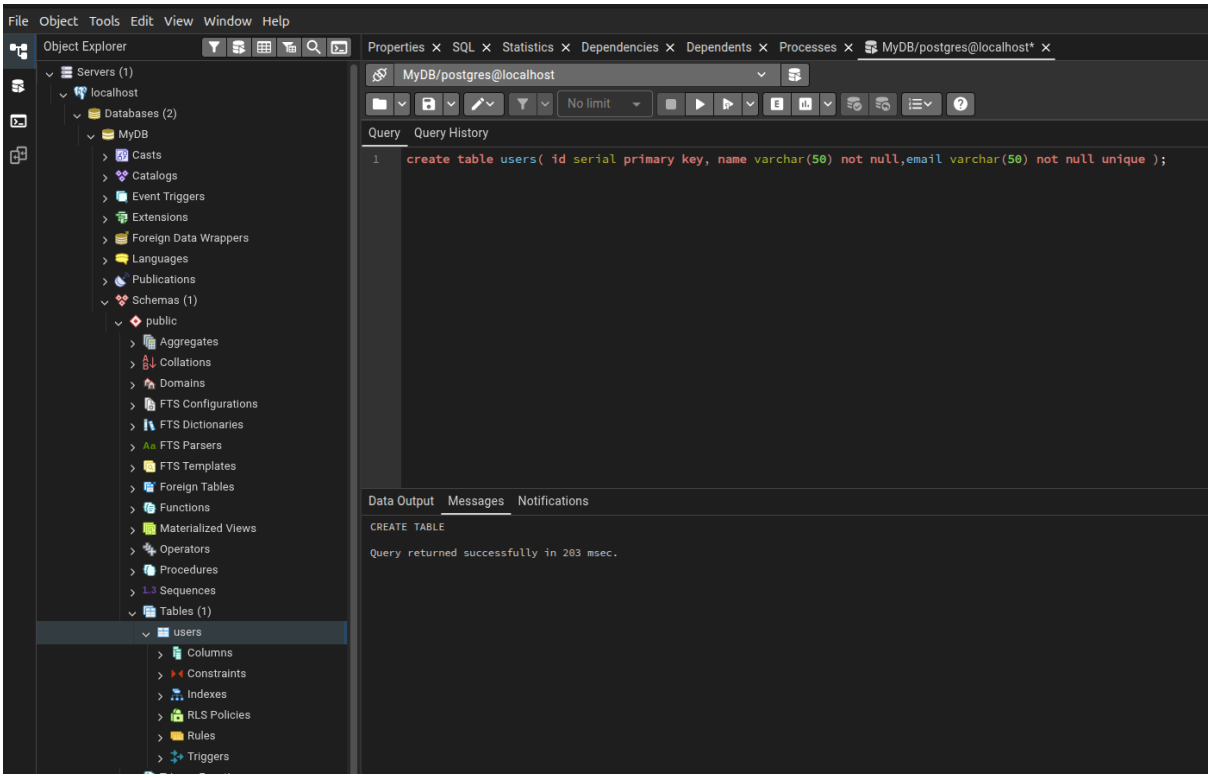
```
/*

 PgAdmin - Query-Tool 7.

create table users( id serial primary key, name varchar(50) not
null,email varchar(50) not null unique );

*/
```


pgAdmin 4
File Object Tools Edit View Window Help
Properties SQL Statistics Dependencies Dependents Processes
Servers (1)
localhost
Databases (2)
MyDB
No dependent information is available for the selected object.
Create
Drop Alt + Shift + D
Drop (Force)
Refresh... F5
Restore...
Backup...
Export Data Using Query...
CREATE Script
Disconnect from database
ERD For Database
Maintenance...
Grant Wizard...
Search Objects... Alt + Shift + S
PSQL Tool
Query Tool Alt + Shift + Q
Properties... Alt + Shift + E
Servers > localhost > Databases > MyDB


File Object Tools Edit View Window Help
Object Explorer
Properties SQL Statistics Dependencies Dependents Processes MyDB/postgres@localhost*
MyDB/postgres@localhost
No limit
Query Query History
1 create table users(id serial primary key, name varchar(50) not null,email varchar(50) not null unique);
Data Output Messages Notifications
CREATE TABLE
Query returned successfully in 203 msec.
Servers (1)
localhost
Databases (2)
MyDB
Casts
Catalogs
Event Triggers
Extensions
Foreign Data Wrappers
Languages
Publications
Schemas (1)
public
Aggregates
Collations
Domains
FTS Configurations
FTS Dictionaries
FTS Parsers
FTS Templates
Foreign Tables
Functions
Materialized Views
Operators
Procedures
Sequences
Tables (1)
users
Columns
Constraints
Indexes
RLS Policies
Rules
Triggers

Server7.js

```
const express = require("express");

const pool = require("./pgdb");

const app = express();

app.use(express.json());


// CREATE

app.post("/users", async (req, res) => {

    const { name, email } = req.body;

    const result = await pool.query("INSERT INTO users (name,
email) VALUES ($1, $2) RETURNING *", [name, email] );

    if (result.rows.length === 0) return res.send("Row Not
Found");

    res.send(result.rows);

});


// READ

app.get("/users", async (req, res) => {

    const result = await pool.query("SELECT * FROM users");

    if (result.rows.length === 0) return res.send("Row Not
Found");

    res.send(result.rows);

});


// UPDATE

app.put("/users/:id", async (req, res) => {

    const id = req.params.id;

    const { name, email } = req.body;

    const result = await pool.query("UPDATE users SET name=
1, email =2 WHERE id=$3 RETURNING *", [name, email, id] );

if (result.rows.length === 0) return res.send("Row Not
Found");

    res.send(result.rows);    });
```

```
// DELETE

app.delete("/users/:id", async (req, res) => {

    const id = req.params.id;

    const result = await pool.query("DELETE FROM users WHERE
id=$1 RETURNING *",[id]);

    if (result.rows.length === 0) return res.send("Row Not
Found");

    res.send(result.rows);

});


app.listen(3000, () => {

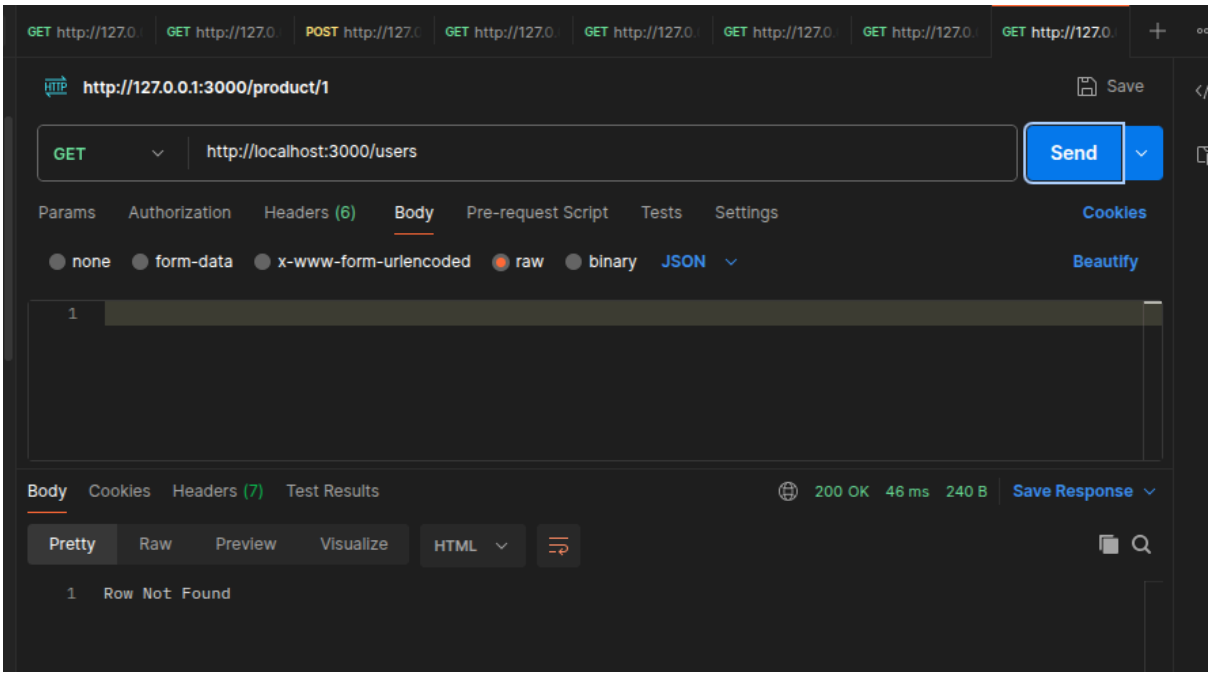
    console.log("Server running:
http://localhost:3000/users");

});
```

```
student@nmamit-nitte:~/MERN$ node Server7.js
Server started on port 3000
```

POSTMAN → Body → raw → Drop Down Select → JSON

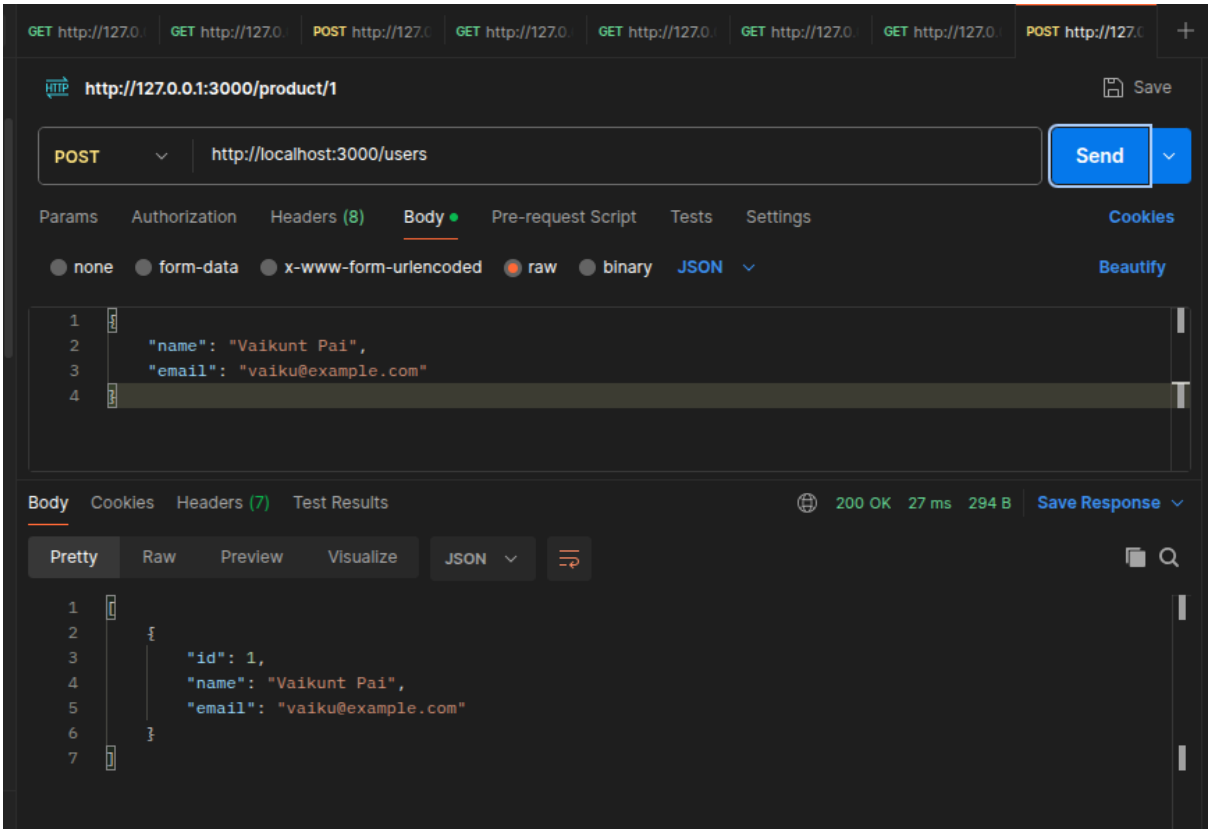
1. http://127.0.0.1:3000/users **Method → GET**

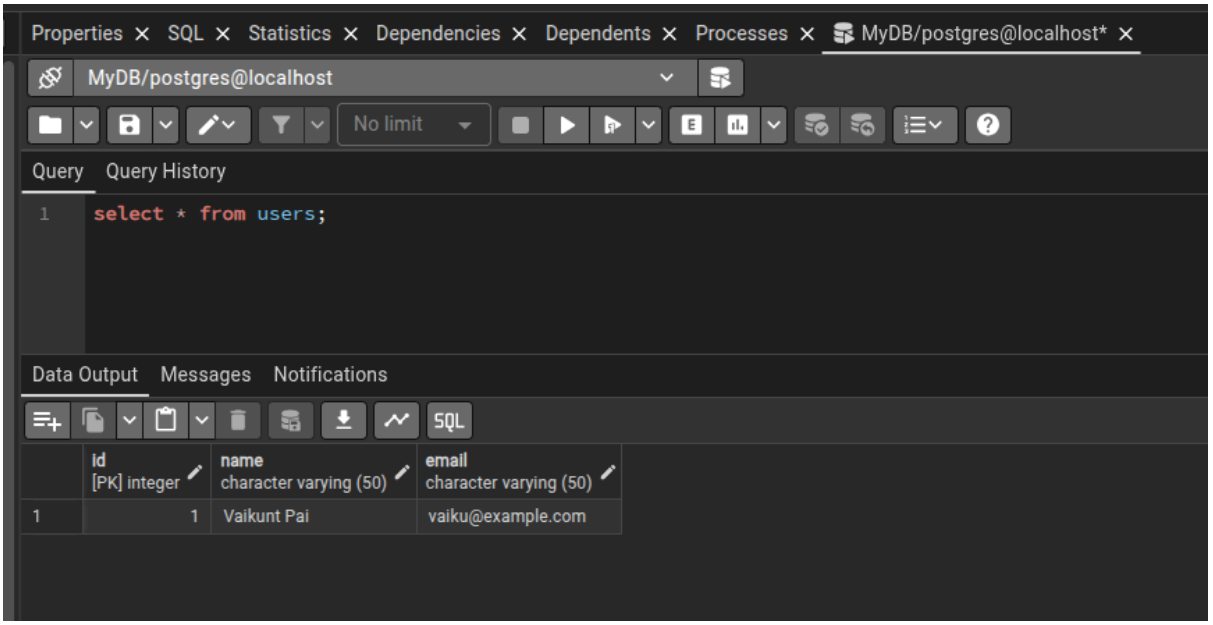


2. http://127.0.0.1:3000/users **Method → POST**

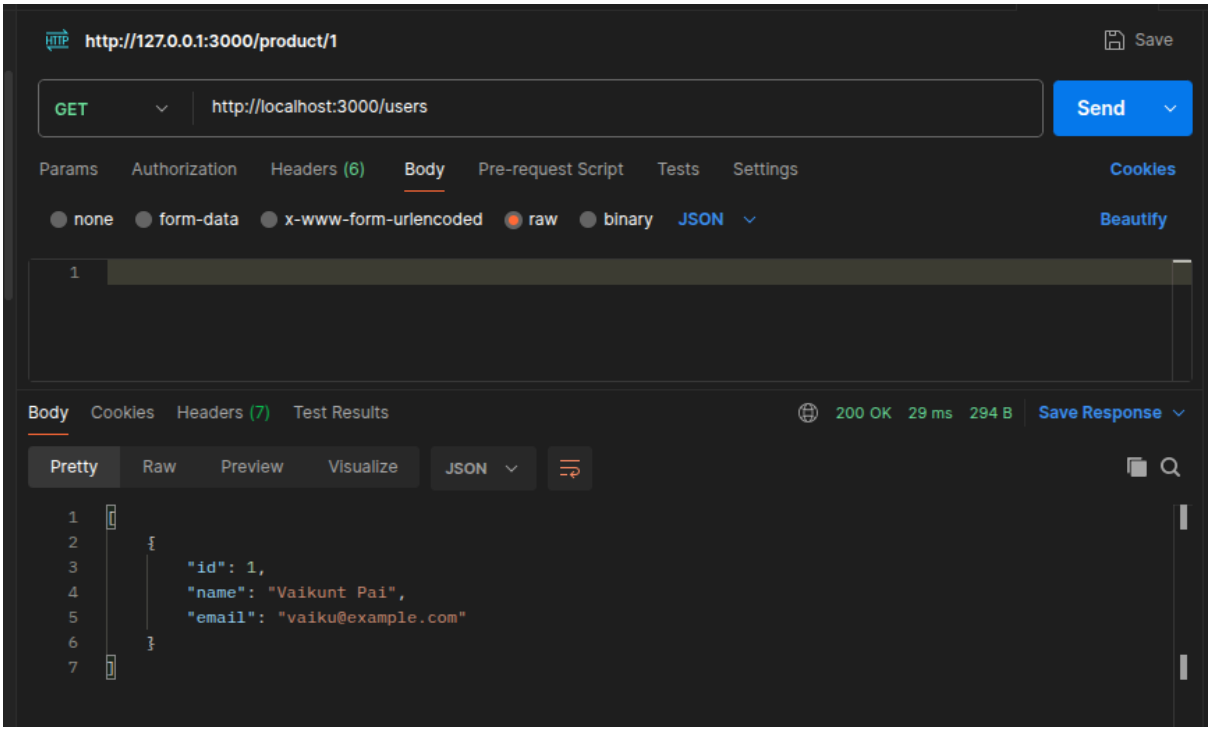
Add the data in Body

```
{
    "name": "Vaikunt Pai",
    "email": "vaiku@example.com"
}
```





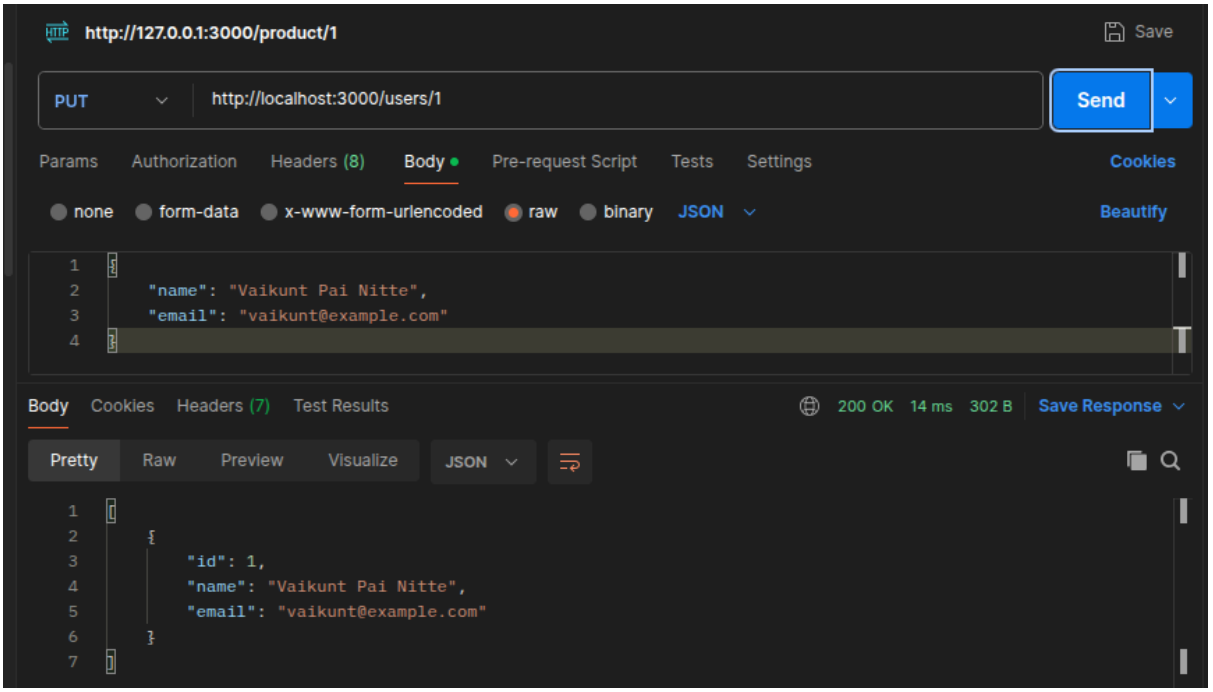
3. http://127.0.0.1:3000/users **Method → GET**



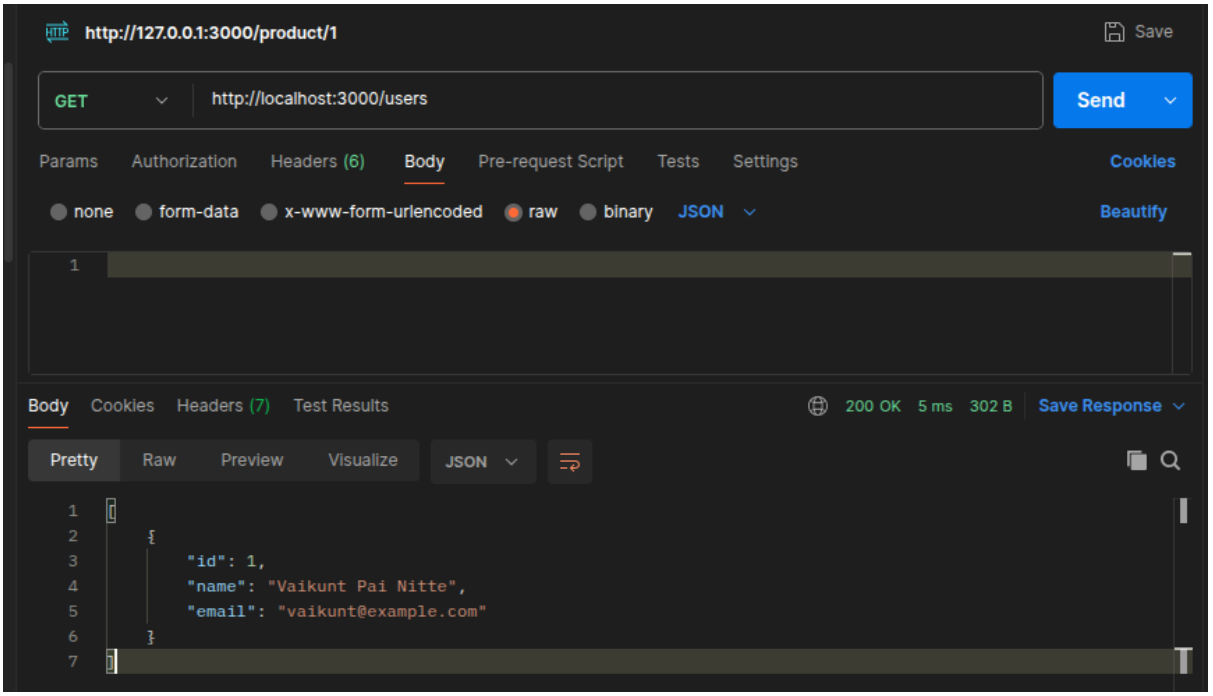
4. http://127.0.0.1:3000/users/1 **Method → PUT**

Add the updated data in Body

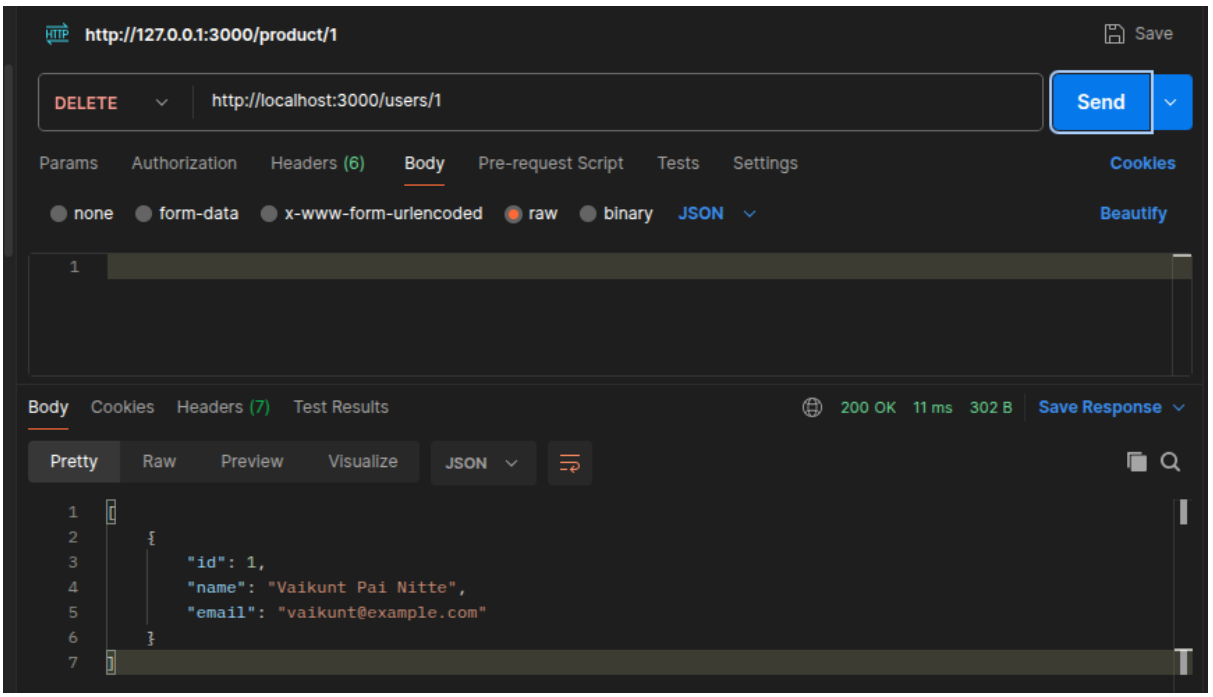
```
{
      "name": "Vaikunt Pai Nitte",
      "email": "vaikunt@example.com"
}
```



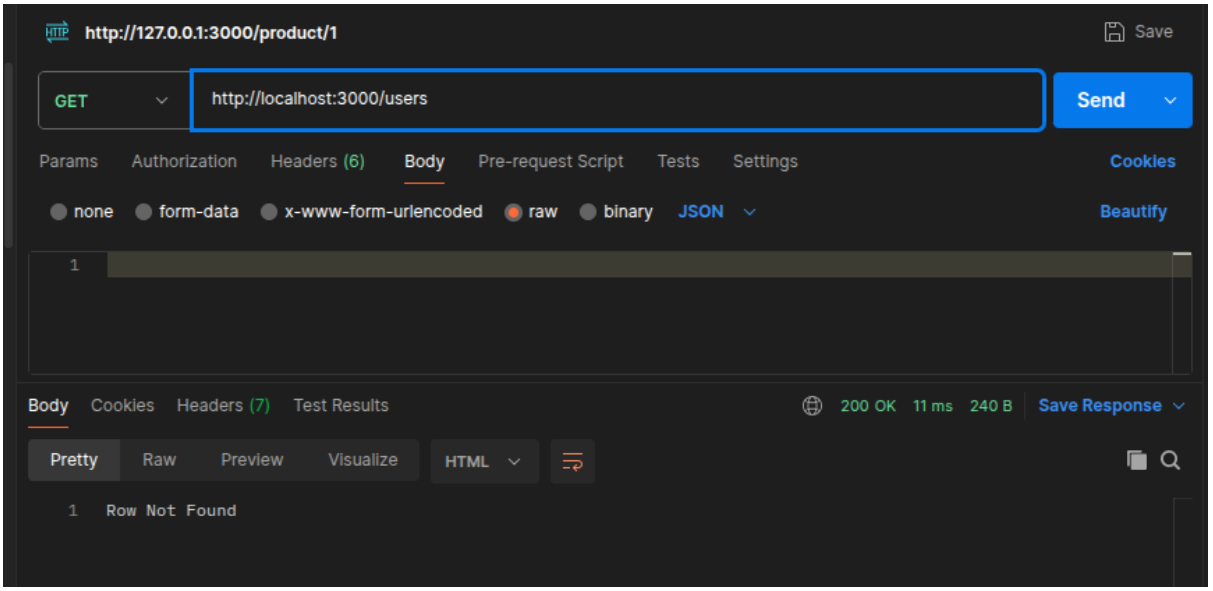
5. http://127.0.0.1:3000/users **Method → GET**

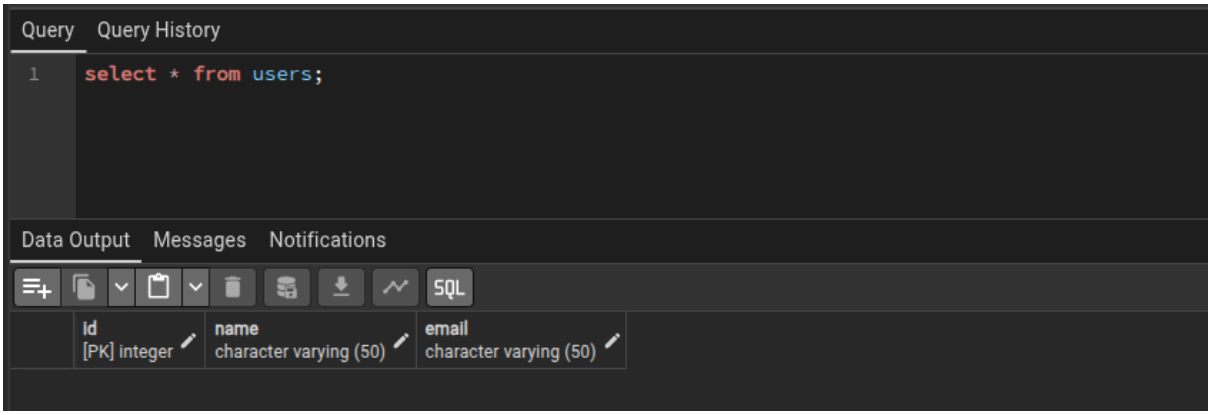


6. http://127.0.0.1:3000/users/1 **Method → DELETE**



7. http://127.0.0.1:3000/users **Method → GET**





Problem 8:

Create a simple REST API using Express.js, MongoDB and Mongoose that allows users to interact with a database. The API should support the following actions:

a. Create: Add a new user to the database.

b. Read: Retrieve all users from the database.

c. Update: Update a user's information in the database(document id)

d. Delete: Delete a user from the database (document id).

The database will store basic user information such as id, name, and email.

Server8.js

```
const express = require("express");

const mongoose = require("mongoose");

const app = express();

app.use(express.json());


async function connectdb() {

    await
mongoose.connect("mongodb://localhost:27017/TestDB");

    console.log("Connected to mongodb");

}

connectdb();


// Without Schema

const UserSchema = new mongoose.Schema({}, { strict: false });

const User = mongoose.model("User", UserSchema);
```

```
//CREATE
app.post("/users", async (req, res) => {
    data = req.body;
    result = await User.create(data);
    console.log(result);
    res.send({ message: "success", result: result });
});

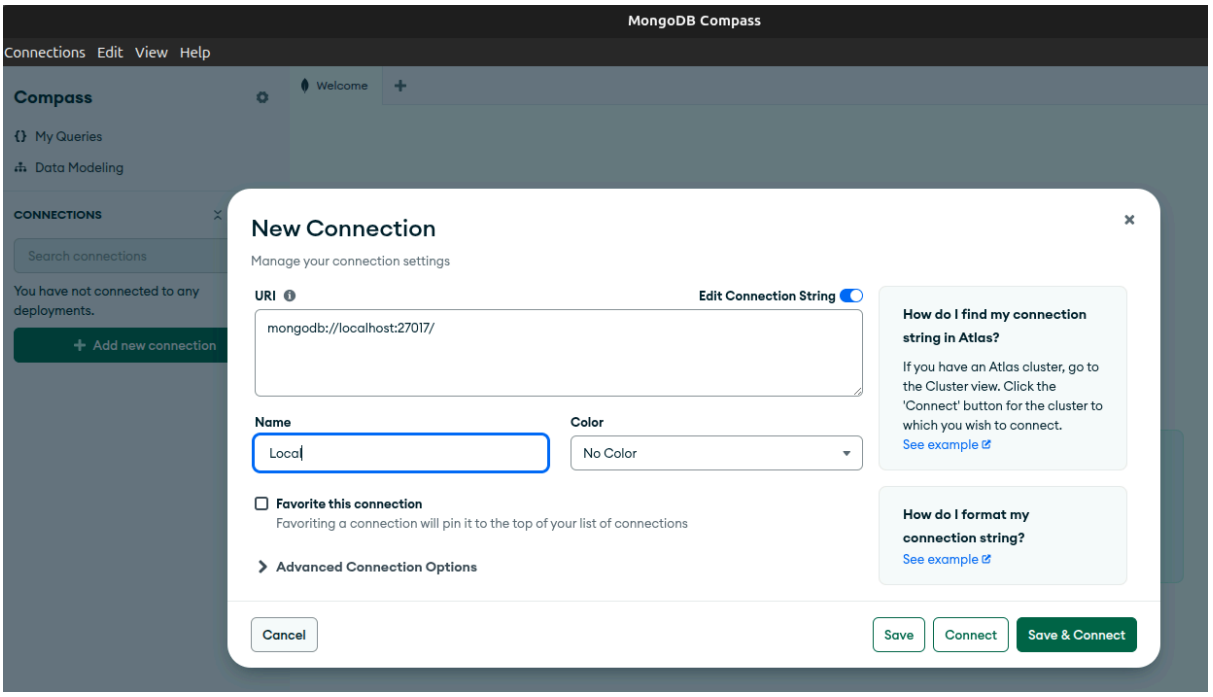
//READ
app.get("/users", async (req, res) => {
    result = await User.find();
    console.log("success", result);
    res.send({ message: "success", result: result });
});

//UPDATE
app.put("/users/:id", async (req, res) => {
    const id = req.params.id;
    const newdata = req.body;
    result = await User.findByIdAndUpdate(id, newdata, { new:
true });
    if (result == null) console.log( "id not found" );
    console.log(result);
    res.send({ message: "success", result: result });
});
```

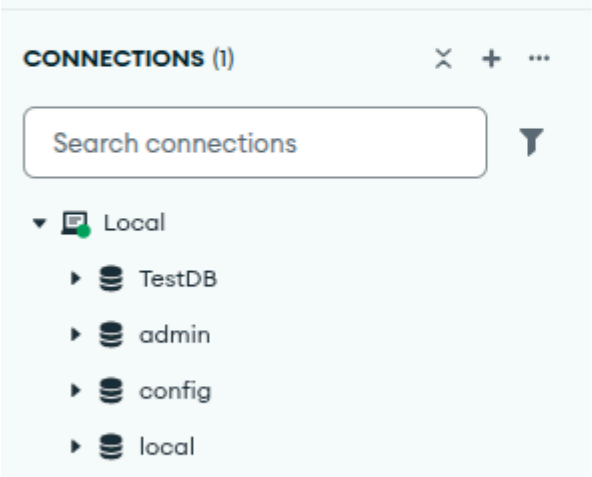
```
//DELETE
app.delete("/users/:id", async (req, res) => {
    const id = req.params.id;
    result = await User.findByIdAndDelete(id);
    if (result == null) console.log( "id not found" );
    console.log(result);
    res.send({ message: "success", result: result });
    // res.send(result);
});


app.listen(3000, () => {
    console.log("Server running:
http://localhost:3000/users");
});
```

Create a connection in MongoDB .. else it will be autocreated

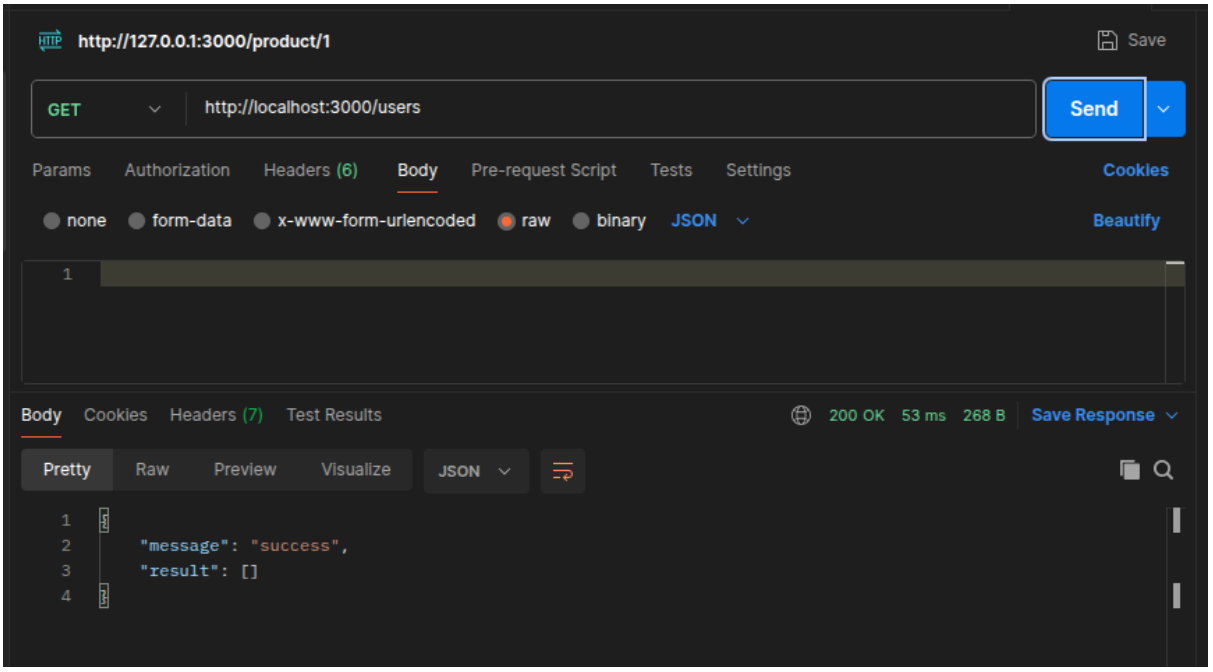


```
student@nmamit-nitte:~/MERN$ node Server8.js
Server started on port 3000
Connected to mongodb
```



POSTMAN → Body → raw → Drop Down Select → JSON

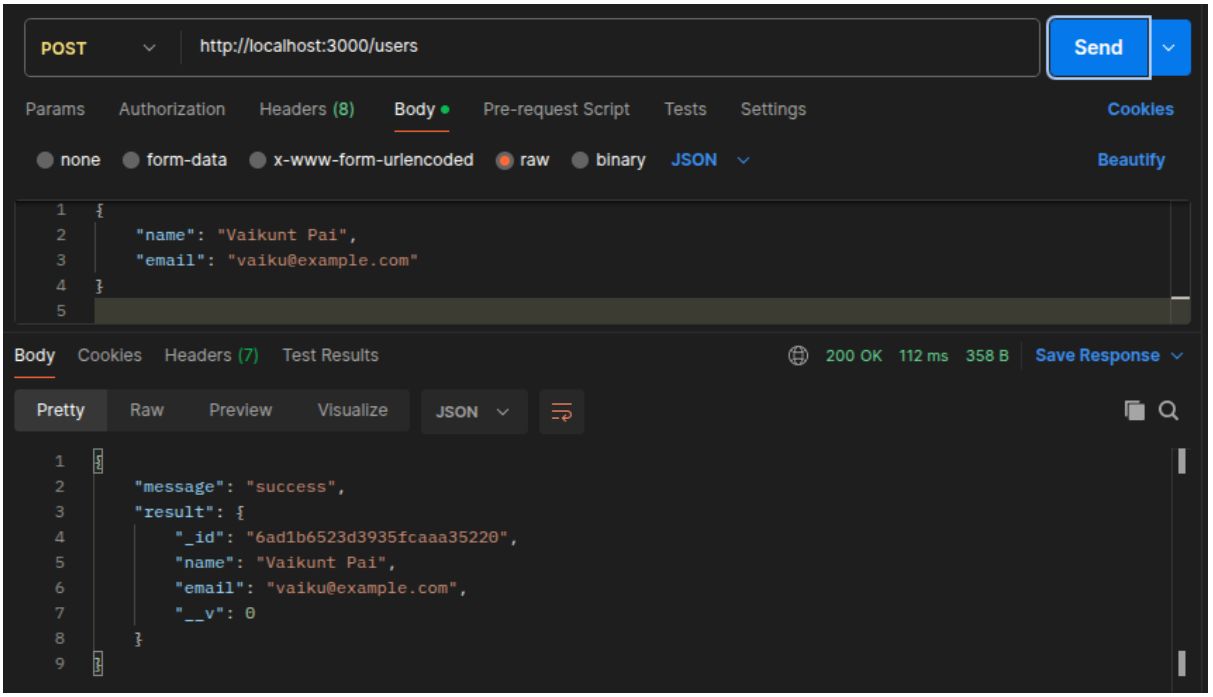
1. http://127.0.0.1:3000/users **Method → GET**

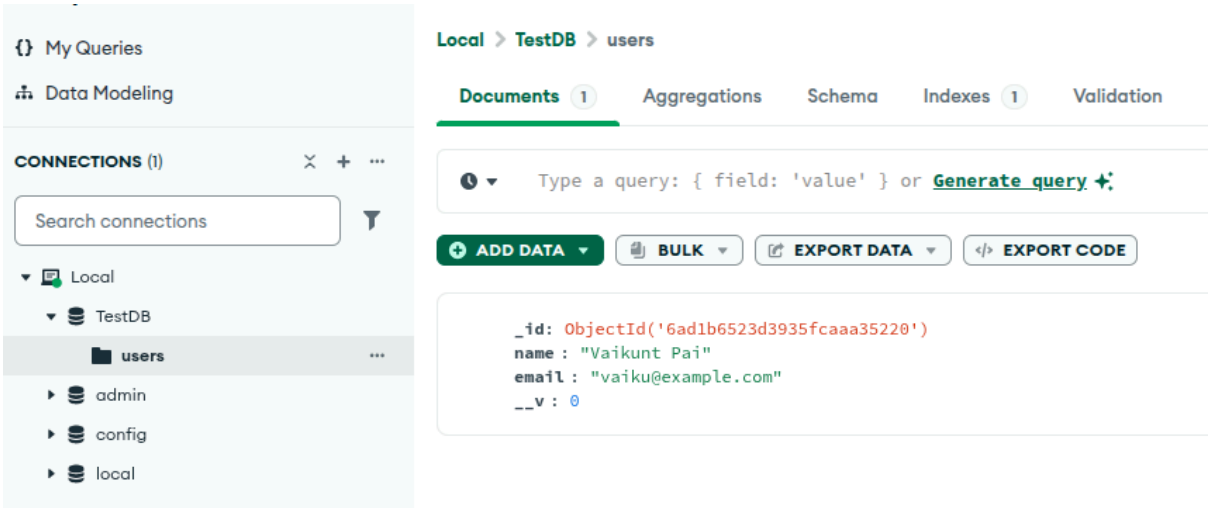


2. http://127.0.0.1:3000/users **Method → POST**

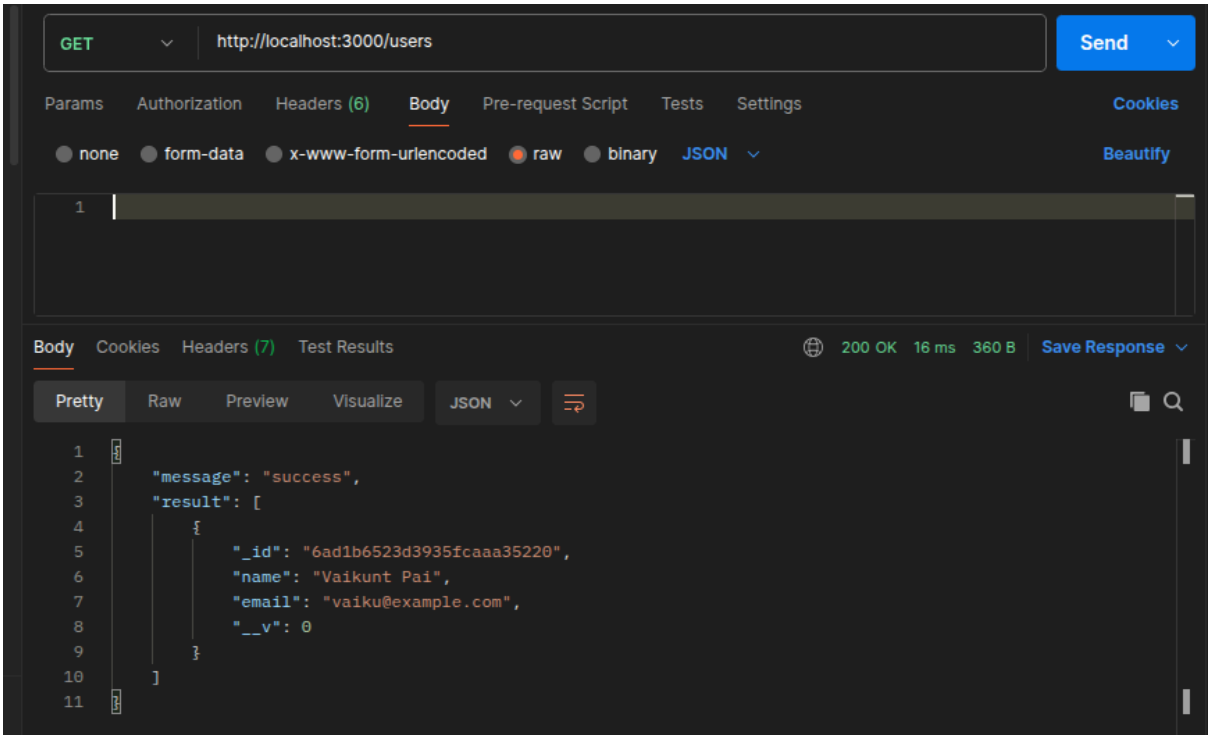
Add the data in Body

```
{
    "name": "Vaikunt Pai",
    "email": "vaiku@example.com"
}
```





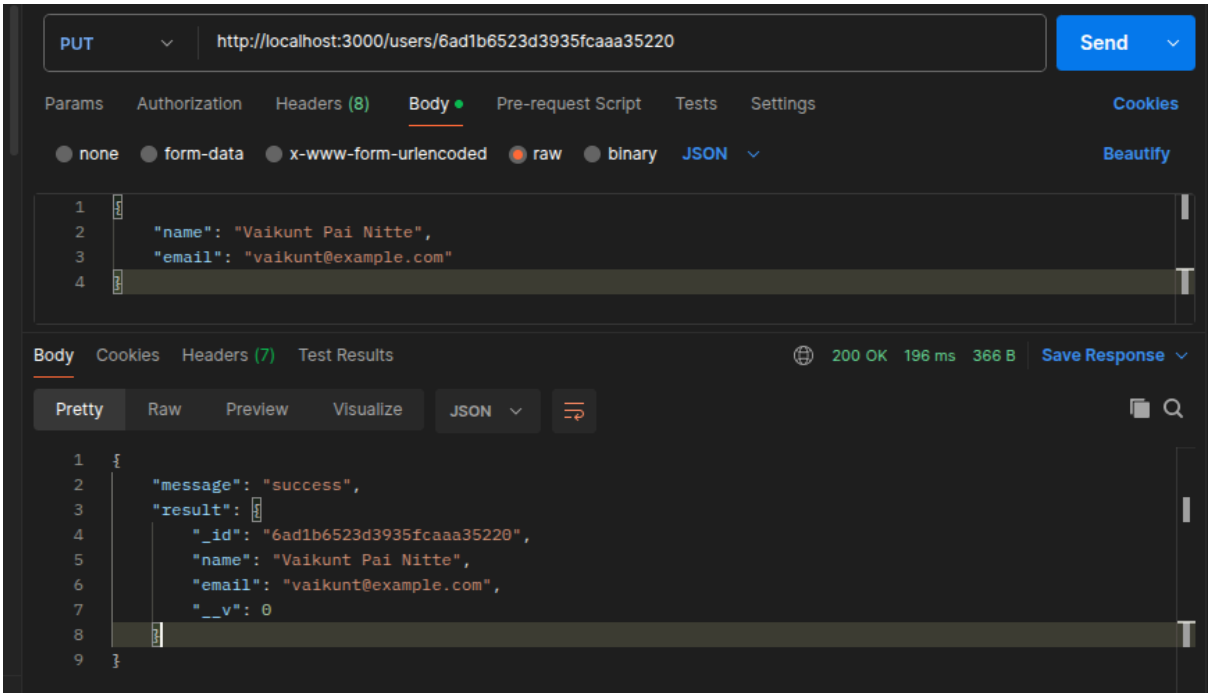
3. http://127.0.0.1:3000/users **Method → GET**



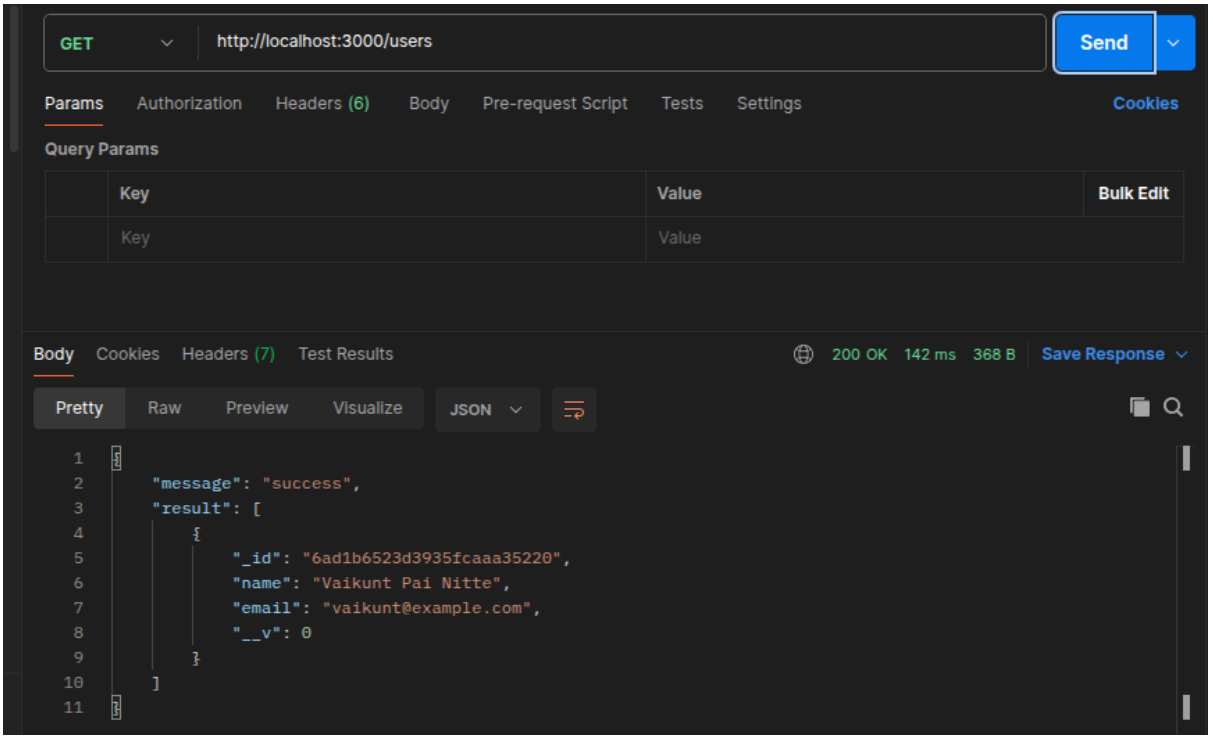
4. http://localhost:3000/users/6ad1b6523d3935fcaaa35220 **Method → PUT**

Add the updated data in Body

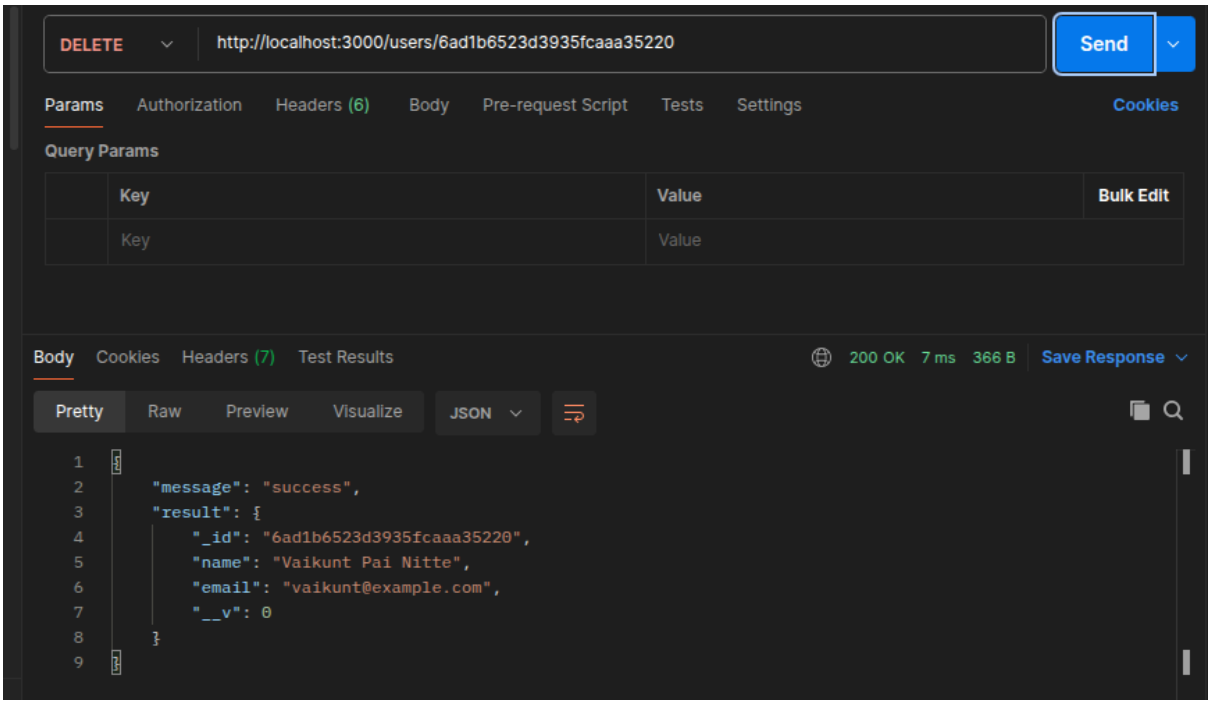
```
{
      "name": "Vaikunt Pai Nitte",
      "email": "vaikunt@example.com"
}
```



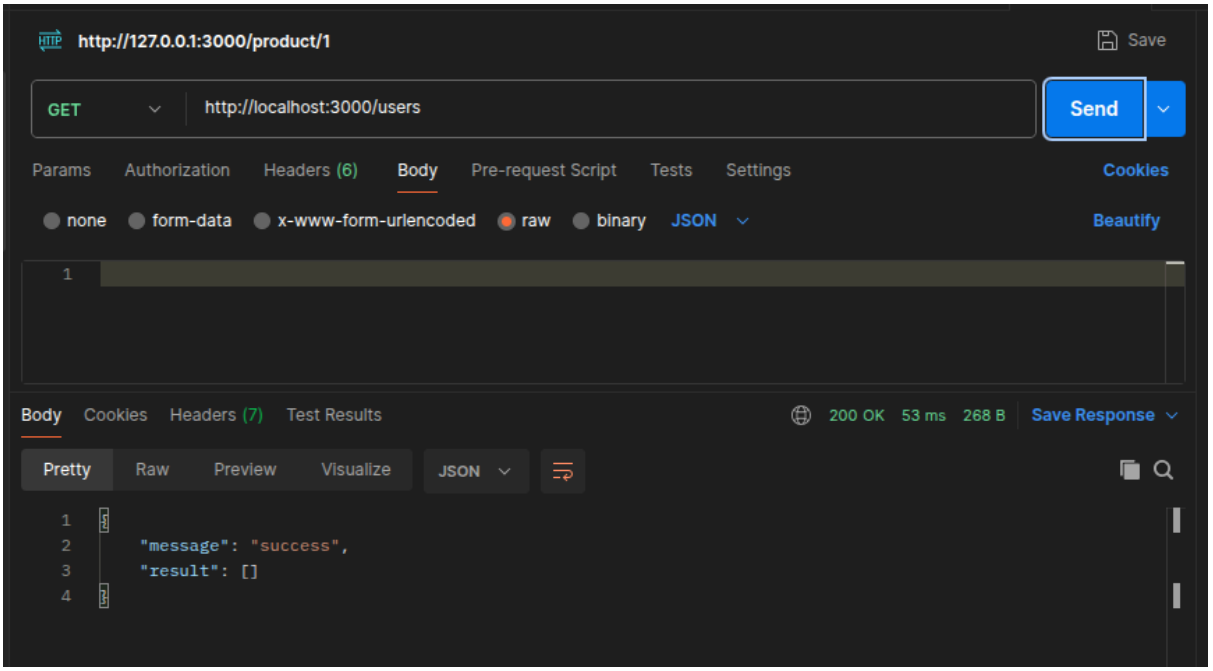
5. http://127.0.0.1:3000/users **Method → GET**

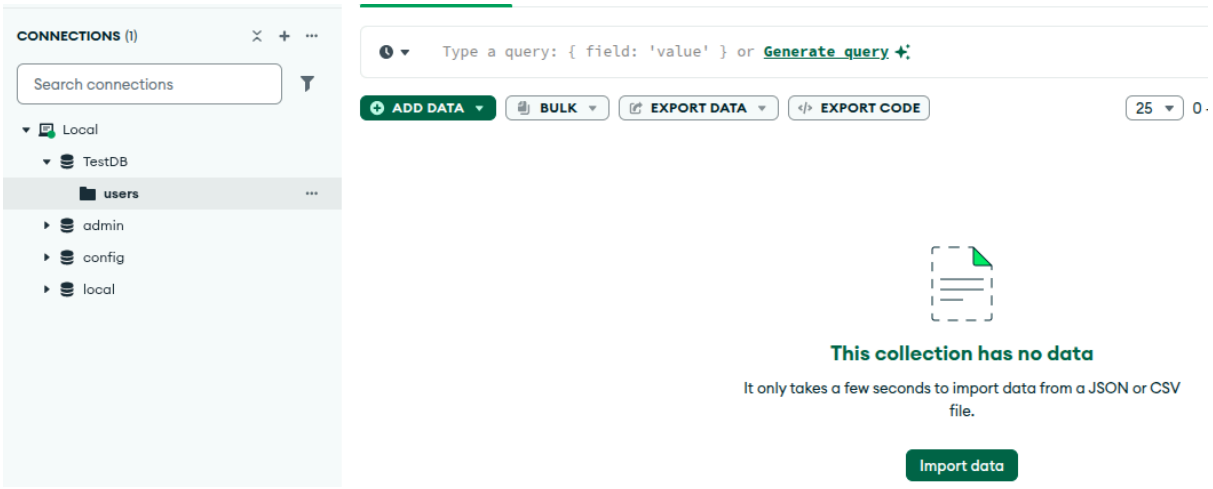


6. http://localhost:3000/users/6ad1b6523d3935fcaaa35220 **Method → DELETE**



7. http://127.0.0.1:3000/users **Method → GET**





Problem 9:

Create a Node.js application using Express.js with global middleware to log all incoming requests in a log file (`requests.log`).

Server9.js

```
const fs=require('fs');

const express=require('express');

const app=express();

app.use(express.json());


app.use('/',(req,res,next)=>{

    const logEntry=`[req.method{req.url}]\n`

    console.log(logEntry.trim());

    fs.appendFileSync("requests.log",logEntry);

    next();

});


function middleware1(req,res,next){

    console.log("Middleware Caught");

    next();

}


app.get('/',middleware1,(req,res)=>{

    res.send("Middleware activated");

})


app.listen(3000, () => {

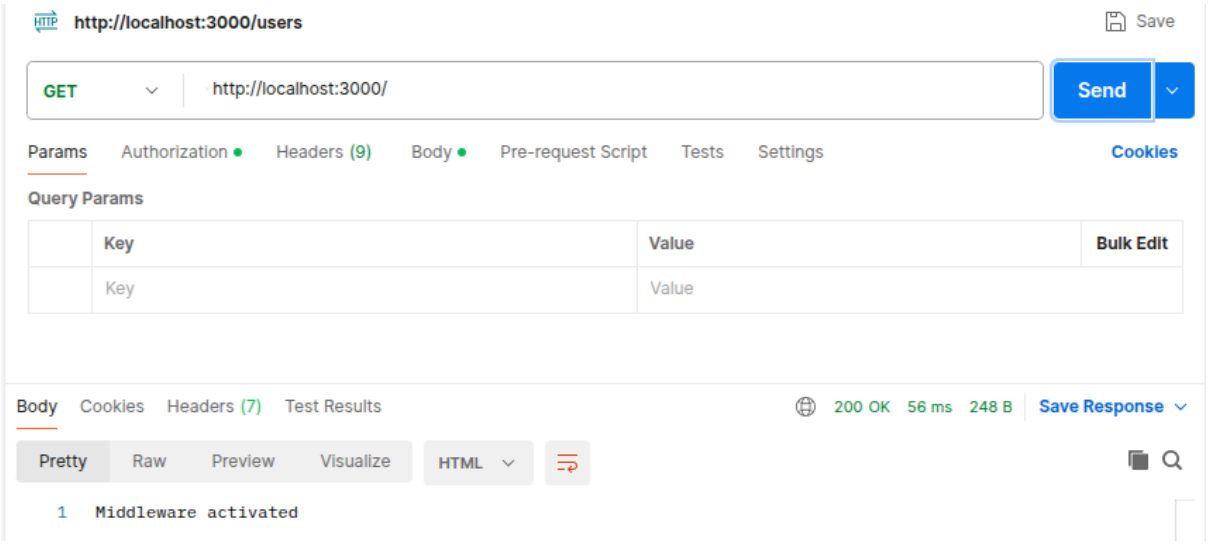
    console.log("JSON server running at
http://localhost:3000/");

});
```

```
student@nmamit-nitte:~/MERN$ node Server9.js
JSON server running at http://localhost:3000/
```

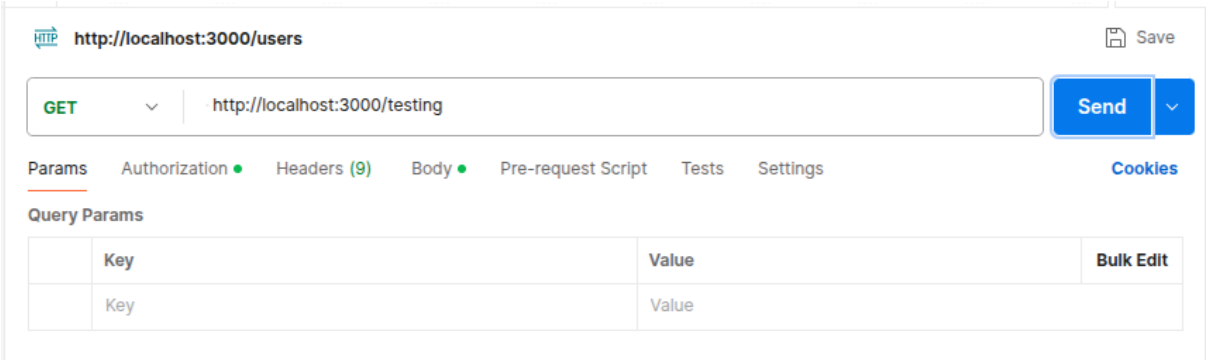
POSTMAN

1. **http://127.0.0.1:3000/ Method → GET**




```
student@nmamit-nitte:~/MERN$ node Server9.js
JSON server running at http://localhost:3000/
[GET/]
Middleware Caught
```

2. **Try with other URL**




```
student@nmamit-nitte:~/MERN$ node Server9.js
JSON server running at http://localhost:3000/
[GET/]
Middleware Caught
[GET/testing]
```

```
student@nmamit-nitte:~/MERN$ node Server9.js
JSON server running at http://localhost:3000/
[GET/]
Middleware Caught
[GET/testing]
student@nmamit-nitte:~/MERN$ cat requests.log
[GET/]
[GET/testing]
```

Problem 10:

Create a simple authentication system using JSON Web Tokens (JWT). You will build the following components:

- a. A Sign Up route to create the user with hashed password.
- b. A login route to authenticate the user and issue a JWT.
- c. A protected route that can only be accessed by users with a valid JWT.

pgdb.js

```
const { Pool } = require('pg');

const pool = new Pool({

user: 'postgres',

password: 'admin',

host: 'localhost',

port: '5432',

database: 'MyDB',

});

module.exports=pool;
```

Server10.js

```
const express = require("express");

const jwt = require("jsonwebtoken");

const bcrypt = require("bcrypt");

const pool = require("./pgdb");


const app = express();

app.use(express.json());


const JWT_SECRET = "anytexthere";

pool.connect(() => {});
```

```
// Signup route

app.post('/signup', async (req, res) => {

    const { username, password } = req.body;

    const hashedPassword = await bcrypt.hash(password, 10);



    try {

        await pool.query('INSERT INTO clients (username,
password) VALUES ($1, $2)', [username, hashedPassword]);

        res.status(201).send("User registered successfully");

    } catch {

        res.status(400).send("Username already taken or
invalid");

    }

});



// Login route

app.post('/login', async (req, res) => {

    const { username, password } = req.body;

    const result = await pool.query('SELECT * FROM clients
WHERE username=$1', [username]);

    const user = result.rows[0];



    if (!user || !(await bcrypt.compare(password,
user.password))) {

        return res.send("Unauthorized Credentials");

    }



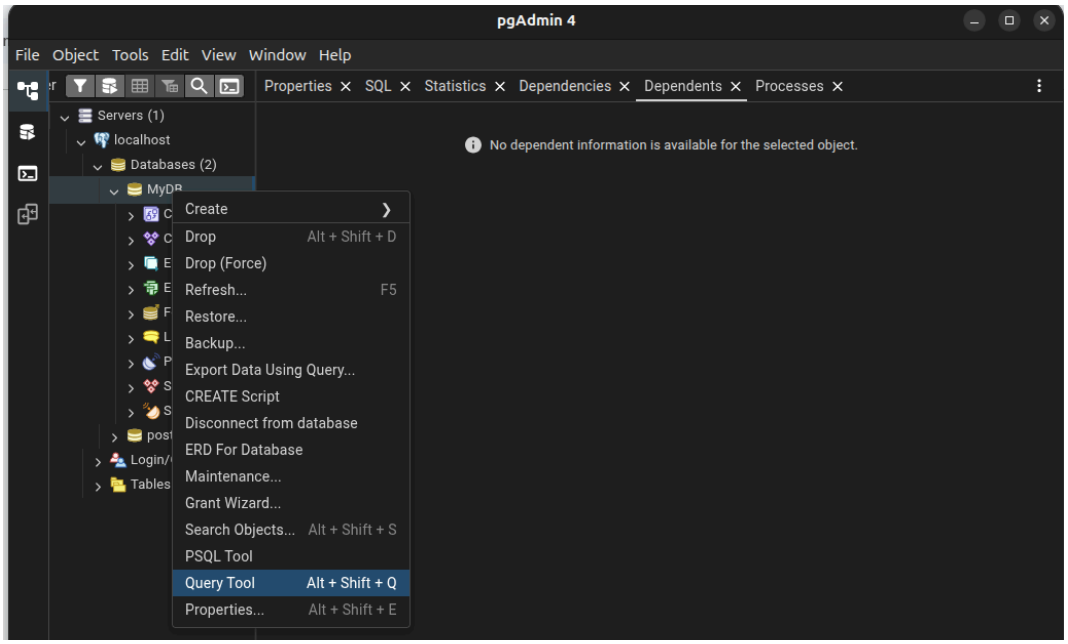
    res.send(jwt.sign({ username }, JWT_SECRET, { expiresIn:
'1h' }));

});
```

```
// Authentication middleware
const authorizeMiddleware = (req, res, next) => {
    const token = req.headers.authorization?.split(' ')[1];
    if (!token) return res.send("Invalid Authorization");
    jwt.verify(token, JWT_SECRET, (err, user) => {
        if (err) return res.send(err);
        req.user = user;
        next();
    });
};

// Protected home route
app.get("/home", authorizeMiddleware, (req, res) => {
    res.send({ message: "Welcome Authenticated User : "+
req.user.username });
});

app.listen(3000, () => console.log("Server running:
http://localhost:3000/"));
```



```
PgAdmin - Query-Tool 7.

CREATE TABLE clients (

    username VARCHAR(100) primary key NOT NULL,

    password VARCHAR(100) NOT NULL

);
```

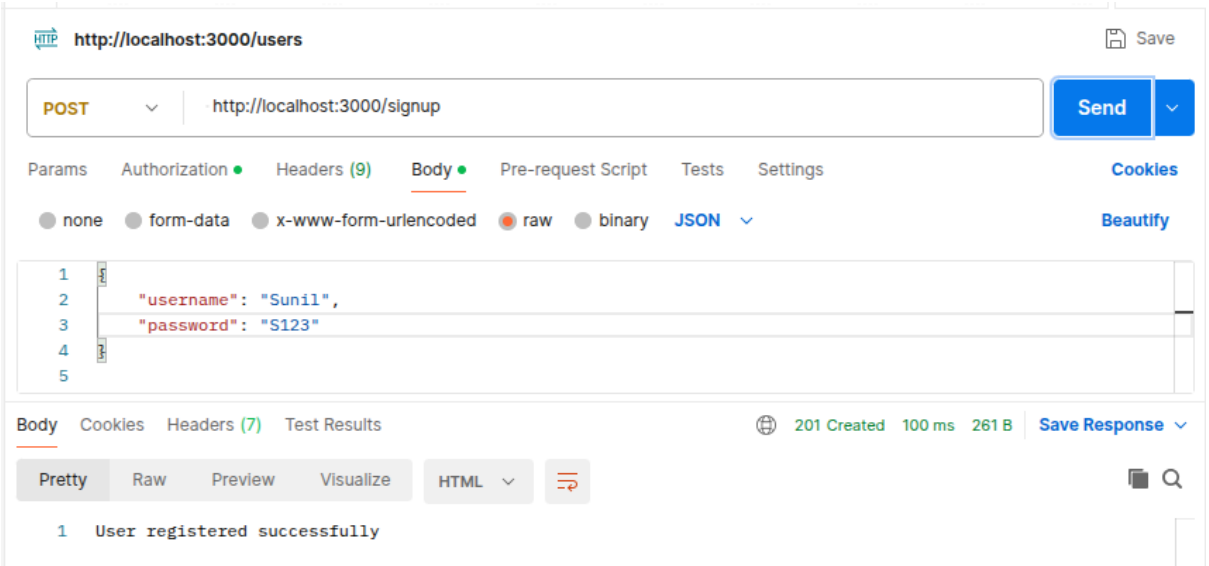
```
student@nmamit-nitte:~/MERN$ node 10.WebToken.js
Server running: http://localhost:3000/
```

POSTMAN → Body → raw → Drop Down Select → JSON

1. http://127.0.0.1:3000/signup **Method → POST**

   Add the data in Body

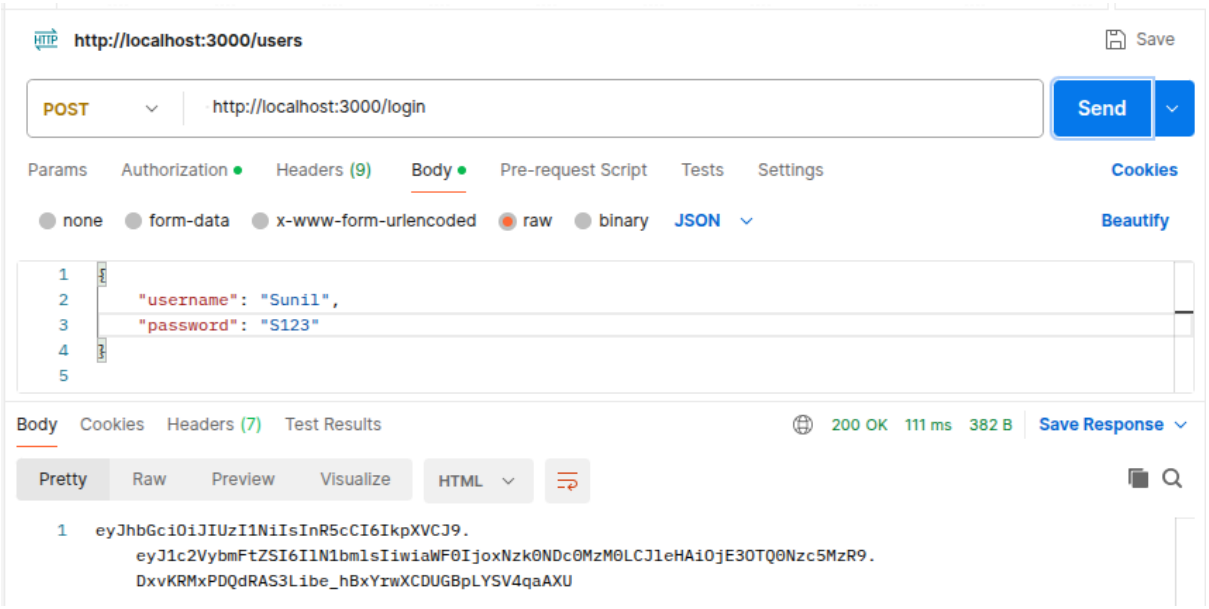
   ```
   {
       "username": "Sunil",
       "password": "S123"
   }
   ```



2. http://127.0.0.1:3000/login **Method → POST**

Add the data in Body

```
{
    "username": "Sunil",
    "password": "S123"
}
```



Copy the Token Key

3. http://127.0.0.1:3000/home **Method → GET**

In Authentication tab → Type Choose → Bearer Token

Paste The Token inside the right side text box

